Enhancing Recommendation Diversity by Re-ranking with Large Language Models

DIEGO CARRARO, Insight Centre for Data Analytics, School of Computer Science & IT, University College Cork, Cork, Ireland
DEREK BRIDGE, Insight Centre for Data Analytics, School of Computer Science & IT, University College Cork, Cork, Ireland

Recommender Systems (RS) should provide diverse recommendations, not just relevant ones. Diversity helps handle uncertainty and offers users meaningful choices. The literature proposes various methods to improve diversity, most notably by re-ranking and selecting from a larger set of candidate recommendations. Driven by promising insights from the literature on how to incorporate versatile Large Language Models (LLMs) into the RS pipeline, in this paper we show how LLMs can be used for diversity re-ranking.

We prompt LLMs to generate a diverse ranking from a candidate ranking using various prompt templates with different re-ranking instructions in a zero-shot fashion. We conduct experiments testing state-of-the-art LLMs from the GPT and Llama families. We compare their re-ranking capabilities with random re-ranking and various traditional re-ranking methods from the literature. We open-source the code of our experiments for reproducibility. Our findings suggest that the trade-offs (in terms of performance and costs, among others) of LLM-based re-rankers are superior to those of random re-rankers but, as yet, inferior to the ones of traditional re-rankers. However, because LLMs exhibit improved performance on many natural language processing and recommendation tasks and lower inference costs, we can expect LLM-based re-ranking to become more competitive soon.

CCS Concepts: • **Information systems → Information retrieval diversity**; **Language models**; **Recommender systems**.

Additional Key Words and Phrases: Recommender Systems, Large Language Models, Diversity, Re-ranking

1 Introduction

Large Language Models (LLMs) have rapidly become a breakthrough technology since the advent of their popular pioneers GPT [49] and BERT [13], introduced in 2018. Since then, many more LLMs with enhanced capabilities have been proposed, such as ChatGPT, Bard and Llama. They can perform various language-related tasks, including, for example, translation, summarization and conversation; and they give the appearance of understanding complex contexts and exhibiting reasoning, planning and problem-solving capabilities [35, 58, 70]. LLMs are typically pre-trained with large datasets of text to serve as general-purpose models and then adapted to different downstream tasks and domains by a variety of approaches [44] — prompting, Supervised Fine Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF) being among the most widely used. Prompting refers to giving the LLM a specific input (e.g. a question, a sentence, a set of instructions), known as a prompt, designed to drive the LLM to generate a desired output. Supervised Fine-Tuning adapts the model by further training the LLM on a task-specific labelled dataset. Reinforcement Learning from Human Feedback (RLHF)

Authors' Contact Information: Diego Carraro, Insight Centre for Data Analytics, School of Computer Science & IT, University College Cork, Cork, Ireland; e-mail: diego.carraro@insight-centre.org; Derek Bridge, Insight Centre for Data Analytics, School of Computer Science & IT, University College Cork, Cork, Ireland; e-mail: d.bridge@cs.ucc.ie.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
© 2024 Copyright held by the owner/author(s).
ACM 2770-6699/2024/10-ART
https://doi.org/10.1145/3700604

Just Accepted

[77] fine-tunes the model using human preference data and is typically used in settings where large amounts of labelled training data are not available or where the domain requirements are human-centred.

Given their abilities and their versatility, researchers in Recommender Systems (RSs) have proposed different ways of integrating LLMs into a recommendation pipeline, for example, to enhance data augmentation, perform feature engineering, design scoring functions, directly provide recommendations, and furnish explanations of recommendations [19, 40]. Some researchers have taken a unified approach, where the entire recommendation task is framed as a text-generation task: a single LLM can perform all the functionalities mentioned above (e.g. [11, 21]), showing how well LLMs can generalise across different recommendation settings.

All the work on RSs we have cited focuses on providing recommendations that are as relevant as possible to a user. However, long before the use of LLMs in RSs, the RS literature recognized that relevant recommendations might not be enough to satisfy a user fully: providing a set of recommendations that is diverse (different from one another) or recommendations that are novel (new or previously unexplored), serendipitous (surprising, unexpected, and going beyond the user's typical preferences) and fair (not discriminating between certain groups of users or items) is also crucial, see, e.g., [31, 59, 65]. Strategies that optimize for these so-called beyond-accuracy objectives have been proposed for traditional RSs, while the topic is under-investigated for LLM-based RSs. The goal of this paper is to contribute to filling this gap.

Our work is a first step that investigates enhancing the top-n recommendation diversity by re-ranking using general-purpose state-of-the-art LLMs. Other works propose to re-rank recommendations leveraging LLMs to improve relevance, but to the best of our knowledge, we are the first ones focusing on beyond-relevance objectives. In particular, our contributions are tailored to the following research questions:

- *RQ1: Can LLMs interpret the building blocks of RS diversification re-ranking, i.e. item diversity and item re-ranking?*
- *RQ2: Given an initial candidate set of items ranked by their relevance to a user, can general-purpose LLMs re-rank these items to increase their recommendation diversity?*
- *RQ3: How do different zero-shot prompts affect the re-ranking performance in terms of relevance and diversity?*
- *RQ4: What are the trade-offs of LLM-based and traditional greedy diversification re-rankers in terms of performance, costs and other important aspects such as control over data, memory footprint and generalisation across domains, among others?*

To answer RQ1, we run an informal preliminary study where, first, we ask ChatGPT to perform some re-ranking tasks, supplementing existing positive evidence already provided in the information retrieval [55] and RS [27] domains. Then, we prompt the model to compare two lists of items in terms of their item diversity. Encouraged by positive findings in these informal studies, where the LLM exhibits re-ranking capabilities and knowledge of item diversity, we then propose a more rigorous methodology to answer RQ2-4.

First, we frame diversification re-ranking as a text generation task in which we prompt an LLM in a zero-shot fashion to generate a more diverse ranking from a candidate ranking. We design eight prompt templates that contain different instructions for re-ranking to explore different ways to approach the generation. We explore a wide range of templates that differ in their complexity, from simple high-level task descriptions to more advanced ones where we provide additional information about the items and further re-ranking guidelines. We also introduce ways of addressing LLM hallucinations (i.e. where the model produces invalid recommendations), a common problem that is usually ignored by most of the LLM literature on RSs.

Then, we run experiments on an anime movie dataset and on a book dataset where a Matrix Factorization RS optimized for relevance provides candidate recommendations. We apply our proposed LLM-based diversification re-ranking to these recommendations using state-of-the-art LLMs. Specifically, we use proprietary models from

Just Accepted

OpenAI[1] (namely, ChatGPT and InstructGPT) and also open-source models from Meta's Llama family[2] (namely, Llama2-7B-Chat and Llama2-13B-Chat). We assess to what extent recommendation diversity improves and how relevance is affected (RQ2, RQ3). We compare with a random re-ranker and some traditional state-of-the-art greedy strategies that intelligently re-arrange items to balance relevance and diversity.

To address RQ4, we run a comprehensive analysis beyond performance for both approaches, where we discuss their trade-offs in terms of costs, control over data, and memory footprint, among other aspects.

Our results demonstrate that, in both datasets, the LLM-based re-rankers we proposed can interpret the re-ranking task, sacrificing some relevance to improving diversity. While LLM-based re-rankers achieve better relevance/diversity trade-offs than random re-ranking, they are still inferior to traditional re-rankers, particularly in relevance-aware metrics, due to issues such as drawing lower-ranked items and the presence of random recommendations. OpenAI's models, particularly ChatGPT, outperform Meta's models, such as Llama2-13B-Chat, aligning with existing literature that credits OpenAI models with more advanced capabilities. The effectiveness of different prompt templates varies, indicating the need for tailored prompt design, with feature-aware templates generally performing better. Finally, the traditional greedy approach is faster and less resource-intensive, making the costs and constraints of LLM-based re-ranking a significant barrier for practical use. Nevertheless, we posit that LLM-based re-ranking has considerable promise. For example, more advanced prompting and fine-tuning are possible directions for making LLMs more competitive when it comes to enhancing diversity. In any case, we would also expect LLM-based re-ranking to become more competitive as LLMs capabilities improve as larger architectures and better human-aligned model versions are released and as the costs of using them continues to fall.

The remainder of this paper is structured as follows. Section 2 reviews related work; Section 3 presents both the preliminary study and our LLM-based re-ranking approach; Section 4 describes experiments and Section 5 analyses their results. Section 6 offers a broader perspective on the trade-offs of LLM-based and greedy re-ranking approaches. Finally, 7 wraps up our findings, discusses the limitations of our study and outlines future directions.

2 Related work

2.1 Measuring recommendation diversity

The most common way to measure the diversity of a set of items, such as a set of n recommendations, uses item dissimilarity. In particular, the average [53] or aggregate [76] pairwise distance between items in the set are popular diversity measures of this kind. Different ways of quantifying item distance have been used. In content-based systems, for example, the complement of the Jaccard similarity on sets of features such as genres [61], the complement of the cosine similarity on term vectors [18], and distance in a taxonomy [76] are possible methods. When items have vectors of user ratings, item distance can be computed using the complement of Pearson correlation [61], the complement of cosine similarity [51] or Hamming distance [34]. Item distance can also be computed on the latent item feature vectors in Matrix Factorization approaches [52, 54, 62].

Some researchers have argued that the traditional diversity metrics do not correlate well with users' perceptions and have proposed alternatives, especially when item features are based on item genres. For example, these new metrics also account for coverage of item feature values, redundancy and the size of the recommendation list [60].

There are also metrics that consider both diversity and relevance at the same time: they quantify the trade-off between diversity and relevance with a single metric. α-NDCG, for example, was introduced by [10] to evaluate information retrieval systems, where the score of a retrieved document is penalized if it shares features with documents ranked higher in the list of retrieved documents. This metric has also been used in RS research in [6, 33]. Inspired by α-NDCG, Vargas & Castells weight a diversity score with item relevance, under the assumption

[1]https://openai.com/
[2]https://llama.meta.com/

Just Accepted

that diversity on irrelevant items does not play a major role in satisfying a user [61]. They designed a framework that allows one to define several metrics that account for a diversity and relevance trade-off based on a ranking discount. In our experiments, we use genres as item features, and we employ their framework and some metrics from it, such as α-NDCG, Expected Intra-List Diversity, and relevancy-aware Subtopic Recall; see Section 4.4.

Finally, for completeness, we note that metrics have also been defined for other types of diversity, such as temporal diversity metrics that capture the extent to which recommendations generated for the same user differ over time, or metrics that take a system-centric, rather than user-centric, perspective by analysing how recommendations differ across users. We refer to [31] and [7] for comprehensive surveys of diversity-aware metrics.

2.2 Increasing recommendation diversity

As well as *measuring* diversity, researchers have proposed ways of *increasing* the diversity of a set of recommendations. Diversification methods can be categorised into two main groups: diversity modelling and diversity post-processing.

The first group of works includes recommendation models that are directly optimized to produce diverse recommendations. Most of these RSs are derivations or extensions of the popular and effective Matrix Factorization (MF) model. For example, Shi et al. [52] designed an objective function that balances the expected relevance and variance of the recommendation list. The variance level is obtained from the latent factor vectors of an MF model and correlates with a user's spectrum of tastes. In other words, the latent factors of a user who rates diverse items have higher variance compared to a user who rates similar items; the former user will consequently receive more diversified recommendations from the model than the latter user does. Another example is [28], which modified the learning function of a pairwise learning-to-rank approach (for implicit feedback datasets) to account for item dissimilarities. A third example is [54], which proposed a revised pairwise learning-to-rank model that works at the item set level (rather than the individual item level): their model is trained by comparing item sets using both relevance and diversity criteria to establish the pairwise ranking.

The second group of works includes approaches that apply a post-processing step (typically, using a greedy strategy) that refines an initial list of recommendations from a conventional RS to obtain a final set of recommendations that balances relevance and diversity. More formally, these techniques typically produce a final list of recommended items RL of size n from a larger set of candidate recommendations CL of size m (with $m > n$), where CL is generated by a baseline recommendation algorithm that ranks items based on their relevance. The vast majority of the RS literature on this uses greedy re-ranking, which involves an iterative process where items are selected from CL one by one and added to RL. At each step, the greedy algorithm selects the candidate item that maximizes a re-ranking objective function f_{obj}, like the one defined in Eq. 1. Here, f_{obj} is a linear combination of two scores, i.e. a candidate item's relevance and the diversity this item brings when added to the current version of RL. In particular, in the equation, $rel(i)$ denotes the relevance score of the item i to a user. $div(i, RL)$ denotes the diversity of RL when the item i is included in RL and it typically makes use of a distance function between items. Finally, the hyperparameter $\lambda \in [0, 1]$ controls the trade-off between the influence of relevance and diversity during the re-ranking.

$$f_{obj}(i, RL) = \lambda \cdot rel(i) + (1 - \lambda) \cdot div(i, RL) \tag{1}$$

Greedy re-ranking has been widely adopted in the RS literature because it is easy to implement and integrate into existing recommender system pipelines. Indeed, its relevance $rel(i)$ and diversification $div(i, RL)$ components can be adapted based on the application domain, and their trade-off can be explicitly controlled with λ.

Just Accepted

Maximal Marginal Relevance (MMR) [5] is one of the first diversification techniques of this kind. It was originally proposed for the information retrieval domain but later adopted in the RS literature, e.g. [14, 62]. In MMR, $div(i, RL)$ is computed as the negative of i's maximum similarity to items already in RL.

In a case-based recommender scenario, Smyth & McClave implemented $rel(i)$ as the similarity between the user's query and an item or case i, while $div(i, RL)$ was the average pairwise distance between i and the items or cases already in RL, the distance being the complement of their similarity [53]. In a conversational recommender scenario, Kelly & Bridge took $rel(i)$ as the predicted item's relevance and $div(i, RL)$ is again the average pairwise distance between i and the items in RL, but this time using the normalized Hamming distance of the two items' binary rating vectors [34].

A more recent stream of research arose from Vargas' work [59], which proposed greedy re-ranking strategies inspired by Intent-Aware (IA) diversification from the field of information retrieval [1]. In essence, IA methods in information retrieval seek to optimize the relevance and diversity trade-off by retrieving documents that are relevant to the user's query but which also cover different aspects of the query (e.g. different meanings of an ambiguous query). In the RS domain, IA methods seek to retrieve items that are predicted to be relevant to the user while covering different aspects such as item feature values or user tastes. For example, Vargas presented explicit Query Aspect Diversification (xQuAD) [62] and its variant Relevance-based xQuAD (RxQuAD) [63]. xQuAD focuses on maximizing the probability that a user will *select* at least one of the recommended items by building an RL that covers all the different user aspects. RxQuAD introduces a relevance model into xQuAD to maximise relevance rather than the probability of choosing a single item.

There are also works that perform post-processing but do not use greedy re-ranking. Instead, they typically aim at solving optimization problems to find the optimal ranking for balancing relevance and diversity; some examples are [29, 51, 71].

In the experiment described in this paper, we use MMR, xQuAD and RxQuAD as our greedy re-ranking baselines against which we compare the performance of our LLM-based re-rankers (see Section 4).

2.3 LLMs and recommender systems

Broadly speaking, work on using LLMs in the RS domain can be categorized into *supporting recommendations* or *directly providing recommendations*.

Representative examples from the first group of works use an LLM to augment training data, extract features, and explain recommendations. Borisov et al. [4] exploit LLMs to generate synthetic examples from tabular data; Liu et al. [42] prompt an LLM to craft news summaries, user profiles, and synthetic news for a news recommender; Mysore et al. [45] propose an LLM-based pipeline that converts user-item interaction data, commonly used to train collaborative filtering systems, into datasets to train narrative-driven recommenders. Inspired by the Retrieval-Augmented Generation (RAG) paradigm [20], Di Palma [15] advocates a comprehensive approach, which he calls a Retrieval-augmented Recommender System, which combines the strengths of retrieved information about items with LLMs to enhance the recommendation task.

When used as a feature extractor, an LLM is typically fed with textual user features, item features or user-item interaction data (e.g. user behaviour or reviews) and returns vectorized embeddings that can be used by a traditional recommender model (often a neural network) to perform various recommendation tasks. Features are extracted, for example, from code snippets [50], from news [72], from tweets [73], and from documents [43], and applied to enhance cross-domain recommendation [16, 25, 26].

As they can generate fluent and coherent text, LLMs are also powerful tools used to provide explanations in natural language for different kinds of recommendations, e.g. for products in e-commerce [8, 11], for leisure products [38], and for music, video games and food [24].

Just Accepted

Works in the second group use LLMs to provide recommendations directly, either by generating item suggestions or by giving scores to user-item pairs. When LLMs generate item suggestions, they can be in the form of item names [11, 37] or identifiers [21, 47]; and the LLM can make next-item recommendations [21, 64] or it can rank a set of candidate items [11]. When LLMs generate scores, they can be in the form of predicted ratings [21, 36], click-through rates and conversion rates [11], or preferences [3, 75].

Other works, different from the two groups we have reviewed so far, are more related to re-ranking recommendations, akin to what we propose in this paper. We briefly review them in the following and we draw explicit comparison with our work in Section 3.2. Dai et al. [12], Li et al. [39] and Hou et al. [27] all use several models from the ChatGPT family to rank an unordered candidate set of items in a zero-shot scenario for domains such as movies, books, music and news, additionally leveraging historical user interactions in the prompts. Sun et al. [55] investigate the capabilities of ChatGPT and GPT-4 at re-ranking candidate documents retrieved by a traditional information retrieval system (in a zero-shot generation and by distilling a specialized model). Suzgun et al. [56] use LLMs for textual style transfer: the system first generates a set of candidate texts in the target style (by zero-shot or few-shot prompting with GPT2) and then re-ranks them according to three types of scores provided by different LLMs. All these works show promising results for LLM-based re-ranking (especially for ChatGPT) but they all struggle to outperform state-of-the-art competitors. Moreover, there is no consistent methodology for evaluating LLM-based re-rankers (taking into account, e.g., invalid recommendations). In Section 3.2, we will compare in detail our work with this literature, highlighting similarities and differences for each component of our system.

3 Recommendation diversification using LLMs for re-ranking

Our work investigates recommendation diversification by using an LLM to re-rank a set of pre-computed candidate recommendations. We frame this task as a zero-shot text generation task and discuss more advanced approaches in Section 7. For re-ranking, we use and compare two state-of-the-art families of LLMs, i.e. ChatGPT/InstructGPT and Llama-2-Chat, which, using SFT and RLHF, have been fine-tuned from their foundation models (GPT and Llama respectively) to follow instructions.

3.1 A preliminary study

Although there is extensive evidence that both ChatGPT and Llama-2-Chat can successfully perform many different tasks, to the best of our knowledge, little is known about whether they are good at re-ranking and whether this can be done in the context of diversity in RSs. The closest works to ours that offer some positive evidence in these regards are [12, 39], which explore ranking an unordered candidate set of items; and [55], where the task is re-ranking a candidate list of items previously ranked by a retrieval system. These works focus on ChatGPT only. To the best of our knowledge, there is no published research to assess whether LLMs are capable of reasoning about item diversity. Thus, we begin our informal investigations by answering RQ1. More specifically, we find answers to the following:

Q1 Can LLMs interpret and perform re-ranking tasks?

Q2 Do LLMs have knowledge about item diversity?

We note that this preliminary study is rather informal and is not a comprehensive evaluation of an LLM's re-ranking capabilities. Instead, it serves as an indication of their potential capabilities towards our larger goal.

To answer Q1, we designed three simple prompts asking the LLMs to re-rank a list of the titles of 20 anime movies. (We use anime movies for this preliminary study because they come from one of the datasets we use in the more substantial experiments described in Section 4.) Specifically, we ask the LLMs to re-rank the lists so that they are ordered alphabetically, or by popularity, or by release date. We expect that the complexity of the tasks should affect the re-ranking performance: when re-ranking by popularity and release date, an LLM must leverage

Just Accepted

additional details about the items to accomplish the task, whereas alphabetical re-ordering can be done by using the items' titles only.

In Appendix A.1, we show three representative examples of the prompts and ChatGPT's outputs. (Llama-2-Chat's outputs are similar, so we do not show them.) As these examples demonstrate, ChatGPT can perform the re-ranking tasks with different degrees of accuracy. In particular, alphabetical re-ranking (Q1-P1) is the most accurate, with only one item incorrectly ranked. By contrast, popularity (Q2-P2) and release date (Q3-P3) re-ranking are only partially consistent with the ground-truth rankings. We ran several examples of this kind (with different item lists to re-rank). We compared the ChatGPT outputs with ground-truth rankings. We obtained scores that vary between 0.5 and 0.8 for popularity re-ranking and between 0.35 and 0.7 for release date re-ranking, according to the rank-biased-overlap metric [66].

To answer Q2, we designed one prompt that asks the LLM to indicate which of two lists, each composed of 10 anime titles, is the more diverse. We deliberately do not provide any definition of diversity in the prompt so that the LLM must interpret its meaning for itself. Similarly to Q1, we built examples that exhibit different degrees of complexity: an "easier" prompt that compares list A that covers one genre (e.g. all items are comedies) and list B that covers multiple genres; and a "harder" prompt that compares two lists that both cover multiple genres. Again, we ran several examples of this kind (with different item lists to re-rank).

In Appendix A.1, we show two representative examples of the prompts and ChatGPT's outputs. Q2-P1 is the easier prompt, while Q2-P2 and Q2-P3 are the harder prompts. (Again, Llama-2-Chat's outputs are similar, so we do not show them.) In each example, ChatGPT's output first includes its interpretation of the meaning of diversity (i.e. a genre-based definition), followed by the candidate lists augmented with the genres it associates with each item in the list, concluding with the answer to the prompt's question along with an associated explanation.

As expected, for Q2-P1, ChatGPT correctly indicates that list B is the more diverse, and it justifies its answer from a genre-based diversity perspective, i.e. the number of different genres a list covers.

For Q2-P2, ChatGPT indicates that list A is the more diverse and again justifies its answer from a genre-based diversity coverage perspective. According to the genres that ChatGPT assigns to the lists, its answer is correct (list A covers 15 genres, and list B 13 genres). However, if we extract ground-truth genres from the dataset that we use in Section 4, then list B is more diverse (list A covers 18 ground-truth genres, and list B 19).

Finally, For Q2-P3, ChatGPT indicates that list B is the more diverse (for the same reason above). However, ChatGPT's conclusion does not correspond with the genres that it assigns because, in fact, it assigns 11 genres to list B against 12 for list A. The incorrectness of the answer is also confirmed by the ground-truth genres we extracted from the dataset, i.e. 19 for B and 24 for A.

This preliminary study suggests that an LLM can re-rank lists of items and can provide insights into their diversity, using its embedded knowledge (without any additional fine-tuning), albeit imperfectly. Moreover, the LLM showed some degree of reasoning when performing the tasks and was able to provide highly structured answers. Correctness also seems correlated with task difficulty.

Again, we emphasis that this preliminary study is not rigorous, and its findings are not measures of LLM abilities. In particular, we loosely defined "correctness" when evaluating answers to Q2. What we take from this informal investigation is an indication that it is worth proceeding to the more rigorous study that we present in the remainder of this paper.

3.2 Prompt-based zero-shot diversification re-ranking

Following the previous section's largely positive findings, we propose a methodology to answer the research question RQ2 presented in Section 1, i.e. whether LLMs can enhance recommendation diversity by re-ranking. We consider the greedy re-ranking framework presented in Section 2.2, where the main components are the candidate list CL, the final recommendation list RL, a relevance function rel defined over the items in CL, a

Just Accepted

```
<Instructions>:
You are given a ranked recommendation list of {m} items for a
user, delimited by triple backticks.
Your task is to re-rank this candidate list and provide a final
top-{n} recommendation list where the goal is to {goal string}.
Strictly use the following format for the output, and don't
provide additional information.

<Output format>:
1-> <item name>
2-> <item name>
...
{n}-> <item name>

<CL>:
```
1. {candidate item name ranked #1 in CL} {additional_features}
2. {candidate item name ranked #2 in CL} {additional_features}
...
{m}. {candidate item name ranked #m in CL} {additional_features}
```
```

Prompt 1. Generic re-ranking prompt template. Tags in angular brackets delimit sections (but they do not appear in the final prompt), and placeholders for data are enclosed in braces.

Just Accepted

diversification function *div* defined over the items in CL and the list RL, and a hyperparameter λ to control the relevance/diversification trade-off. We also assume the candidate list CL is produced by a baseline recommender. In Section 4, we choose a Matrix Factorization model to provide CL for our experiments, but in practice this can be done by any recommender that can produce a ranked list of items.

We frame the re-ranking task as a text generation task to be accomplished by an LLM in response to specific prompts that request the production of RL from CL. The prompt template comprises three sections, see Prompt 1. The first section of the prompt provides the context and instructions the LLM must follow to generate RL. The second section provides the output format the LLM must adhere to. The third section provides the candidate list CL that the LLM must re-rank. However, except for the use of triple backticks to delimit CL, we do not explicitly mark-up the three sections: the tags in Prompt 1 are for the purposes of exposition in this paper, they are not included in the final prompt. The decision to exclude them is in line with what the literature suggests (e.g. [12, 21, 55]). We will now discuss each of the three sections in more detail.

Clear instructions are crucial to obtaining the desired output since the LLMs we use are aligned with high-quality instruction-based prompts. We designed eight subtypes of the generic template, each identified by a different re-ranking goal; see Table 1. Each goal maps to a different λ value, with T1, T4-T7 aiming at equally balancing relevance and diversity (i.e. $\lambda = 0.5$ in traditional greedy re-ranking approaches) and T2, T3, T8 aiming at maximizing diversity only (i.e. $\lambda = 0$). The related research literature proposes alternative ways to model different re-ranking trade-offs. Li et al. [39] address the trade-off between recommendation relevance and fairness by explicitly instructing the LLM to select a specific number of candidate items from different groups of items — in their case, popular and unpopular items. Suzgun et al. [56] use a combination of scores from different LLMs

Table 1. Mapping of placeholders in the generic prompt template.

Template id	λ	{goal_string}	{additional_features}
T1	0.5	"balance relevance and diversity"	None
T2	0	"maximize the items' diversity in the list"	None
T3	0	"maximize the items' genre-based diversity in the list"	None
T4	0.5	"balance relevance and genre-based diversity"	None
T5	0.5	"balance relevance and diversity"	List of the candidate item's genres
T6	0.5	"balance relevance and genre-based diversity"	List of the candidate item's genres
T7	0.5	"balance relevance and diversity. Guidelines to perform the re-ranking are: Use the plot summary information of each item attached in curly bracket"	Candidate item's plot summary
T8	0	"maximize the books' diversity in the list. Guidelines to perform the re-ranking are: Use the plot summary information of each item attached in curly bracket"	Candidate item's plot summary

to adjust the trade-off between textual similarity, target style strength, and fluency for arbitrary textual style transfer.

The prompt template also includes placeholders {m} and {n}. These are replaced with the size of *CL* and *RL*. We explain how we choose the values of m and n in Section 4.

The prompt template specifies an output format to guide the LLM's generation, simplify output parsing (i.e. extracting the final ranking from the LLM's answer), and reduce its ambiguities. Similarly to [55], we opted for a numbered list (with numbers representing an item's rank, i.e. its relevance in *CL*), followed by the string `->`, to help to identify item names when parsing the output. To the best of our knowledge, the literature offers no advice about how to represent ranked lists in a prompt. There have, however, been works in the literature that represent un-ranked candidate lists in prompts, e.g. by identifying items with letters [12] or by simply appending items one after another [39].

Finally, the prompt must include the list *CL*. We represent its items with their names, e.g. anime titles or book titles (see Section 4), similar to what is done in most of the literature (although some works use item identifiers, e.g. [21]). Items are also numbered in our prompts to report their rank (i.e. their relevance). Also, for templates T5, T6, T7 and T8, which we call feature-aware templates, we replace the `{additional_features}` placeholders to augment an item with its genres (T5, T6) and its description (T7, T8), attempting to equip the LLM with explicit item features to leverage for re-ranking. (In our experiments in Section 4, we use genres extracted from the datasets and descriptions extracted from prompting ChatGPT.) Feature-aware templates can be seen as a form of RAG-based prompting, where each prompt is augmented with some extra contextual information. The only difference between traditional RAG and our approach is that traditional RAG systems typically retrieve this additional context on-the-fly (i.e. when generation occurs), while we retrieve the whole set of item features in advance of re-ranking (see Section 4). In Appendix A.2, we show an example of a prompt (i.e. after the placeholders in the template have been instantiated). We also show ChatGPT's output.

Templates T1-T8 explore different degrees of complexity and help us in addressing RQ3. However, several other prompting techniques might have been explored — for example, few-shot prompting (i.e. providing task-related demonstrative examples), or Chain-Of-Thoughts (CoT) prompting (i.e. where the LLM is driven to provide

Just Accepted

Table 2. Statistics of the datasets

	# ratings	# users	# items	avg. # of ratings per user	avg. # of ratings per item	# genres	avg. len. of item titles	sparsity
Anime	17M	118k	2.6k	143	6.4k	40	20.6	94.50%
Goodreads	8M	166k	8k	50	1k	16	37.1	99.40%

the requested output through intermediate steps) [41, 67]. However, the main goal of this study is to shed light on the capabilities of LLMs to understand the diversification re-ranking task and produce meaningful recommendations rather than finding the prompting strategy that performs best. The exploration of further more advanced LLM-based re-ranking strategies is left as future work.

Finally, in our prompt-based formulation, we do not explicitly model f_{obj}, nor the rel or div functions, in the prompts. We instead 'implement' f_{obj} by including key terms such as "relevance" and "diversity" (or "genre-based diversity") for the rel and div functions, respectively, and terms such as "balance" and "maximize" to model the λ trade-off. However, we posit that our prompt-based implementation is a raw form of greedy re-ranking. This is likely because LLMs such as ChatGPT and Llama2 are autoregressive models that generate text (i.e. the final list RL) in an iterative way, i.e. item by item until RL is composed of n elements.

4 Experiments

In this section, we outline the experimental methodology and the datasets we used in our experiments. We have open-sourced the code for reproducibility.[3]

4.1 Datasets and preprocessing

In our experiments, we use the Anime Recommendation Database[4] and the Goodreads Book Graph Datasets[5]. The former contains about 17 thousand anime movies, and ratings from about 325 thousand users. The latter contains about 2 million books, and ratings from about 876 thousand users. Both datasets include item genres that we use as item features.

We apply the following preprocessing procedure to prepare the datasets for the experiments. We remove items (and their ratings) whose genres are unknown. For the Anime dataset, we also remove movies whose titles are not written in the Roman alphabet and whose release date is from 2021 onwards to increase the chance that LLMs have been exposed to data about these movies. This is not necessary for the Goodreads dataset because the books all have titles in the Roman alphabet and were published before 2018. We then map ratings into a 1 to 5 scale and filter out users with fewer than 70 and more than 300 ratings. Table 2 reports the statistics of the datasets after the preprocessing.

4.2 Methodology

The following methodology applies to both datasets. We randomly split the dataset into training and test sets. We use a user-based split, with 80-20% proportions. We train a Matrix Factorization baseline recommender (which we denote with MF) on the training set, using the RankSys implementation[6] of the factorization algorithm that is proposed in [48]. We then randomly sample 500 test users and their data to run the experiments with the following procedure.

[3]https://github.com/cdiego89phd/chat-reranking
[4]https://github.com/Hernan4444/MyAnimeList-Database
[5]https://mengtingwan.github.io/data/goodreads.html
[6]https://github.com/RankSys/RankSys

Just Accepted

For each test user, we use the MF baseline to compute a ranked candidate item list CL, such that none of the items in CL is in the training dataset (i.e. MF recommends only unseen items). We employ InstructGPT (gpt-3.5-turbo-instruct version) and ChatGPT (gpt-3.5-turbo-0613 version) from the OpenAI GPT family[7], and Meta Llama2-Chat (7B[8] and 13B[9] versions) to re-rank CL as outlined in Section 3.2. We used the proprietary OpenAI API service when re-ranking with InstructGPT and ChatGPT.[10] In the case of the non-proprietary Llama2-7B-Chat and Llama2-13B-Chat, we used two NVIDIA A40 48GB GPUs to host the models and perform re-ranking.

As anticipated, LLMs can return partially invalid outputs. When the output is invalid in fairly simple ways, we try to remedy the situation intelligently. In cases when the LLM appends to the items' titles additional details (e.g. lists of genres, item descriptions, release dates), despite all prompts including clear instructions to outlaw this, we remove such extra content; we also remove any extra content that is prepended or appended to the final recommendation list (e.g. repetition of the re-ranking instructions from the prompt such as "here is your re-ranked list of anime"); and we implement very simple inexact string matching methods (e.g. LLMs tend to add an extra space when encountering semicolons in item titles and our inexact matching takes care of this). But there are cases where invalid output is not so easily remedied, especially when the items themselves are invalid. Examples of invalid items include the following: the output list RL may contain items whose titles did not appear in the candidate list CL; and there may be cases where there is a significant mismatch between the title of the item in CL and its title in RL, e.g. "`Naruto`" instead of "`Naruto:Shippuden`". We discard these items from RL because there is not a straightforward solution to address the invalid generation. In these cases, the RL is incomplete, $|RL| < n$. We fill the list by inserting random items from CL until $|RL| = n$. Instead, we could, but do not, repeat the generation step to try to get a valid list of items: at this stage, we prefer to quantify the problem and leave more advanced solutions for future work.

Prompts T5-T8 presented in Section 3.2 need item features such as genres or item descriptions such as plot summaries. The former are already included in the datasets (see Table 2). The latter are extracted beforehand for each item by prompting ChatGPT with the following prompt:

```
Please provide a one-sentence description of the following item: {item name}.
```

To compare our LLM-based re-rankers, we employ the MMR, xQuAD and RxQuAD greedy re-rankers presented in Section 2.2. We choose these because their re-ranking strategy is explicit, they are fast and effective at diversifying recommendations, and they can use the item genres already available in the datasets as features to re-rank the CL of each user. Note that, unlike the LLM-based re-rankers and the MRR re-ranker, xQuAD and RxQuAD additionally make use of the training data (to calculate aspect probabilities) to produce the re-ranking. In Section 5, we show that this might explain why xQuAD and RxQuAD perform better than the other re-rankers on some metrics. We use the implementations of MRR, xQuAD and RxQuAD that are provided within the RankSys library, mentioned earlier. As a baseline, we also use a Random re-ranker from the same library that randomly samples items from CL to generate RL.

We acknowledge that greedy re-rankers, such MMR, xQuAD and RxQuAD, that rely on heuristics are no longer the state-of-the-art in recommendation diversification. In particular, in large-scale systems, heuristic methods are giving ways to statistical models, such as Determinantal Point Processes (DPPs) [68]. These methods seem to scale better and can out-perform the heuristic methods on relevance and diversity metrics. There would be an even bigger gap between the performance of our LLM-based re-rankers and recommenders using DPP.

[7]https://platform.openai.com/docs/models/gpt-3-5
[8]https://huggingface.co/meta-llama/Llama-2-7b-chat-hf
[9]https://huggingface.co/meta-llama/Llama-2-13b-chat-hf
[10]https://openai.com/index/openai-api/. At the time we ran the experiments, the inference cost for InstructGPT was 1.5$/1M for input tokens and 2$/1M for output tokens; for ChatGPT, the inference cost was 0.5$/1M for input tokens and 1.5$/1M for output tokens.

Just Accepted

There are several reasons why we do not include statistical methods in our experiments. First, our focus is mainly on exploring the *applicability* of LLM-based re-ranking. We know that, without further improvements (larger language models, task fine-tuning, and so on), the performance of LLM-based re-ranking is not likely to rival other methods. Showing the performance of LLM-based re-ranking relative to the performance of MRR, xQuAD and RxQuAD suffices for appreciating the size of the shortfall. Second, MMR, xQuAD and RxQuAD are clearly stated methods with public (and hence reproducible) reference implementations in the RankSys library. The greedy approach even has commonalities with the autoregressive generation strategy of LLMs. None of this can be said for the statistical methods such as DPPs. They would introduce a number of issues, orthogonal to the main thrust of this paper, such as the selection of their hyperparameters and the choice of sampling method.

4.3 Hyperparameter tuning

We use a validation set, a random sample of 20% of the ratings from the training set, to tune the following hyperparameters for each dataset. We set the number k of factors of the baseline recommender MF, choosing k from $\{20, 50, 100, 150\}$ and optimizing for NDCG at cutoff 10. We found $k = 20$ for the Anime dataset and $k = 50$ for the Goodreads dataset. As is common, we set n, the size of the final set of recommendations to 10. Finally, to set m, first we find the mean μ and standard deviation σ of the greatest rank in CL of an item inserted in RL for each re-ranker, i.e. $\max_{\forall i \in RL} \{rank_{CL}(i)\}$; second, we take the largest value among these and set $m = \mu + \sigma$. We found $m = 40$ for the Anime dataset and $m = 50$ for the Goodreads dataset. This means that the values we are using for m are bigger than those we find in the closest works to ours in the literature, e.g. in [27, 55], $m = 20$; in [12], $m = 5$. This will make LLM-based re-ranking in our scenario more challenging because we will have greater prompt lengths and higher inference costs. But, at the same time, our scenario is more realistic because in real-world applications m is unlikely to be as small as 5 or 20.

4.4 Evaluation metrics

To assess the relevance of recommendations, we measured Precision, Recall and NDCG at cutoff 10. However, in Section 5, we report only NDCG because the three metrics reveal consistent findings. To measure recommendation diversity, we employ α-NDCG, Intra-List Diversity (ILD), Expected Intra-List Diversity (EILD), Subtopic-Recall (SRecall) and relevancy-aware SRecall (rSRecall), extensively discussed in [59]. ILD calculates the average pairwise distance of the items in RL; to measure the distance, we use the complement of the Jaccard similarity between pairs of items calculated on the item genres. SRecall calculates the proportion of genres covered by the items in RL. EILD and rSRecall calculate ILD and SRecall on the sublist of relevant items in RL, respectively. α-NDCG is based on NDCG, but it is designed to account for relevance but also coverage of aspects (i.e. genres) and their redundancy; we set $\alpha = 0.5$ following the argument in [59].

5 Results

For ease of exposition, we divide this section into three subsections. In the first, we focus our analysis on comparing LLM-based re-rankers with a random re-ranker (we want to know whether LLM re-ranking is more than arbitrary) and traditional re-rankers (we want to know how LLM re-ranking is positioned with respect to simple state-of-the-art approaches). In the second subsection, we focus our analysis on comparing different prompt templates for LLM-based re-rankers: we want to know how different prompts affect performance. Lastly, in the third subsection we analyse the costs incurred by the different re-rankers.

Tables 11 and 12 in Appendix B report the full set of results presented in this section. In these tables we also report confidence intervals calculated with a 95% confidence level and we use them through our analysis to help outline the uncertainty of our results. Non-overlapping intervals imply statistical significance between performances while, despite common belief, overlapping intervals do not necessarily imply lack of statistical

Just Accepted

Table 3. Re-ranking performance for the Anime dataset. We report the scores achieved by the baseline MF on the six metrics. Then, for each re-ranker, we report the performances in terms of percentage difference with respect to the scores of the baseline MF. We report the average performance across the eight templates for each LLM-based re-ranker and, in parentheses, we report the score of the prompt template that performed best for that particular metric. All metrics are computed with a cutoff of size $n = |RL| = 10$. For all metrics, the higher the scores, the better the performance, and we highlight in bold the best performance for each metric.

	NDCG	**α-NDCG**	**EILD**	**ILD**	**rSRecall**	**SRecall**	
MF	**0.316**	0.325	0.221	0.776	0.234	0.473	
Re-ranker							
Random	-58.8	-52.6	-57.3	4.5	-38.0	5.6	**Diff (%)**
MMR	-5.4	-3.4	**0.4**	**12.6**	1.5	**15.0**	
xQuad	-4.6	**7.9**	-4.2	0.5	**7.5**	14.0	
RxQuad	-6.5	4.1	-6.4	-0.8	4.3	7.0	
ChatGPT	-10.1 (-2.8)	-6.0 (-0.8)	-9.4 (-0.8)	4.7 (5.3)	-2.5 (1.5)	7.8 (9.7)	
InstructGPT	-14.9 (-12.0)	-10.2 (-7.5)	-14.7 (-12.0)	4.7 (5.5)	-6.1 (-4.3)	7.2 (8.8)	
Llama2-7B-Chat	-21.5 (-17.0)	-15.5 (-12.7)	-23.3 (-17.3)	4.6 (5.5)	-10.8 (-7.3)	8.0 (9.1)	
Llama2-13B-Chat	-26.0 (-15.8)	-21.6 (-11.2)	-25.1 (-16.3)	4.4 (5.4)	-14.7 (-7.6)	4.7 (6.2)	

Table 4. Re-ranking performance for the Goodreads dataset. We report the scores achieved by the baseline MF on the six metrics. Then, for each re-ranker, we report the performances in terms of percentage difference with respect to the scores of the baseline MF. We report the average performance across the eight templates for each LLM-based re-ranker and, in parentheses, we report the score of the prompt template that performed best for that particular metric. All metrics are computed with a cutoff of size $n = |RL| = 10$. For all metrics, the higher the scores, the better the performance, and we highlight in bold the best performance for each metric.

	NDCG	**α-NDCG**	**EILD**	**ILD**	**rSRecall**	**SRecall**	
MF	**0.249**	0.271	0.092	0.689	0.355	0.871	
Re-ranker							
Random	-66.3	-59.6	-71.3	2.0	-43.8	2.1	**Diff (%)**
MMR	-4.7	-6.6	**-2.5**	**17.7**	-5.3	7.2	
xQuad	-7.8	**13.7**	-9.3	-8.2	**8.8**	**10.0**	
RxQuad	-8.1	11.9	-8.6	-7.7	7.4	8.6	
ChatGPT	-33.4 (-24.3)	-19.3 (-10.5)	-46.0 (-33.9)	-4.8 (-2.4)	-20.0 (-10.3)	1.8 (3.6)	
InstructGPT	-53.5 (-50.6)	-36.6 (-32.2)	-66.5 (-60.4)	-4.7 (-3.6)	-35.0 (-31.2)	1.4 (2.4)	
Llama2-7B	-59.5 (-55.1)	-50.4 (-46.2)	-70.0 (-62.4)	-1.1 (1.6)	-43.7 (-39.4)	1.1 (2.4)	
Llama2-13B	-61.3 (-54.5)	-48.4 (-37.5)	-71.9 (-66.6)	-2.2 (1.1)	-41.8 (-34.0)	1.5 (2.2)	

significance [22]. In fact, the confidence intervals in our results are quite large for all metrics except for ILD and SRecall — due to high variability in the values of the metrics but also the small number of users in our experiments, i.e. 500. This limits the number of cases where we can be sure of statistical significance (non-overlapping intervals). In future work, we hope to extend our experiments with a larger group of users and that will allow us to perform a deeper statistical analysis and strengthen our findings. This should be possible assuming that the costs (both monetary and computational) of using LLMs continues to fall.

5.1 Comparison of LLM-based re-rankers with random and traditional re-rankers

Tables 3 and 4 report the results of the re-rankers on the Anime and Goodreads datasets, respectively. For each re-ranker, we report the percentage difference with respect to the baseline recommendations from MF. For the

Just Accepted

Table 5. Average lowest rank of items in the candidate list *CL* recommended in the final list *RL*. High values correspond to low ranks. To calculate the lowest rank, we do not consider the random recommendations in *RL* provided to the user when invalid generations occur.

Re-ranker	Anime	Goodreads
MMR	19.5	22.58
xQuAD	21	17.9
RxQuAD	16	15.9
ChatGPT	22.6	31.9
InstructGPT	22.3	37.7
Llama2-7B-Chat	25.5	39.5
Llama2-13B-Chat	28.9	43.1

LLM-based re-rankers, we report the average performance across the eight templates and, in parenthesis, we report the best performance achieved by one of the prompt templates for a particular metric. We postpone the fine-grained analysis across different prompt templates to the next subsection.

As expected, re-ranking decreases relevance and increases the diversity of recommendations for both datasets. In particular, for the Random re-ranker, relevance, measured by NDCG, drops significantly, between 58.8% and 66.3%. Relevance also plays a major role in the α-NDCG, EILD and rSRecall metrics, so we see large decreases in their values for the Random re-ranker. The diversity of its outputs (ILD and SRecall) increases, as typically happens when introducing randomness into recommendations, but only slightly for Goodreads.

In both datasets, all the intelligent re-rankers (both the traditional ones and the LLM-based ones) outperform the Random re-ranker for all the metrics to a great extent (and their performances are statistically significantly different), except for ILD and SRecall (where the superiority is not consistent across all intelligent re-rankers). This confirms what we were looking to prove with RQ2: that LLMs can interpret diversification re-ranking and produce rankings that are more than arbitrary.

Regarding traditional re-ranking, MMR and xQuAD seem to have a better trade-off than RxQuAD, but this strongly depends on which metrics one considers to be more important. RxQuAD has the highest drop in relevance as measured by NDCG in both datasets and, while xQuAD is superior in this metric to MMR for the Anime dataset, the opposite stands for the Goodreads dataset. As expected, MMR is the best for ILD on both datasets since it optimizes this metric. While MMR also improves SRecall on both datasets (even having the best score on the Anime dataset), it fails to improve α-NDCG, and it slightly improves the other relevance-aware diversity metrics (i.e. EILD and rSRecall) on the Anime dataset but not in the Goodreads dataset.

On both datasets, xQuAD is the best re-ranker at improving α-NDCG as this metric is very similar to what it optimises. Similarly, also RxQuAD improves on this metric as expected, but to a lesser extent than xQuAD. xQuAD improves the other diversity metrics too, except for EILD and ILD on both datasets (and it is the best for rSRecall and SRecall on both datasets and for SRecall on Goodreads). Finally, RxQuAD improves α-NDCG, rSRecall and SRecall (as expected, being similar to xQuAD), but in none of these metrics does its scores outperform all the other traditional re-rankers. The lower performance of RxQuAD in terms of relevance-aware metrics is quite surprising since this approach is designed to focus on relevance more than the other two re-rankers. Other works have found similar outcomes [33]: we posit that this is because the datasets exhibit a weak correlation between relevance and item features, in contrast with the assumption made by the re-ranking objective function RxQuAD is built upon.

On the Anime dataset, the average decrease in NDCG for the LLM-based re-rankers (roughly 10-26% loss) is greater than that for the traditional re-rankers (4.6-6.5%), although at its best ChatGPT outperforms all the other

Just Accepted

Table 6. Average percentage of random recommendations (due to invalid generation) provided to the 500 users, distributed across the datasets, the LLM-based re-rankers and the prompt templates. In bold we report the lowest value in each group.

Dataset		Re-ranker			Prompt template							
Anime	Goodreads	ChatGPT	InstructGPT	Llama2-7B-Chat	T1	T2	T3	T4	T5	T6	T7	T8
2.3%	13.1%	3.0%	**1.2%**	18.9%	4.0%	**3.1%**	8.8%	4.8%	9.3%	8.9%	18.0%	22.4%

re-rankers (losing just 2.8%); see the full results in the Appendix. There are similar results for the other metrics, particularly those where recommendation relevance plays a major role (i.e. α-NDCG, EILD and rSRecall), where the best performance of ChatGPT is close and sometimes better than that of the traditional re-rankers. Overall, traditional re-rankers are statistically significantly better than LLM-based re-rankers (except ChatGPT) for NDCG, α-NDCG and EILD. The metric where LLM-based re-ranking does best in comparison with traditional re-ranking is ILD (and, where better, statistical significance is achieved). For this metric, the OpenAI re-rankers perform better than xQuAD and RxQuAD. Finally, there is a clear difference between the LLM-based re-rankers themselves: ChatGPT is superior to all others according to all metrics (except for SRecall where Llama2-7B-Chat has a better average performance than ChatGPT) . Also, OpenAI models are better than Meta ones, with Llama2-13B-Chat being the worst of all the re-rankers.

On the Goodreads dataset, there is a similar pattern of results for the LLM-based re-ranking, but this time the performance gap with traditional re-rankers is higher across all metrics (in particular for NDCG and α-NDCG). ChatGPT is not as close to some traditional re-rankers as it was on the Anime dataset (and differences are statistically significant). It is still the case that the LLM-based re-rankers perform better than xQuAD and RxQuAD for ILD (with a statistically significant difference), and, among themselves, their performance from best to worst remains ChatGPT, then InstructGPT, then Llama2-7B and lastly Llama2-13B.

Three factors might explain why LLM-based re-rankers are inferior to traditional re-rankers in these experiments, particularly for relevance-aware metrics. First, we argue that relevance in *RL* should fall when drawing low-rank items from *CL*, assuming that items in *CL* are accurately ranked by their relevance. Indeed, Table 5 shows that LLM-based re-rankers draw items ranked lower in *CL* than the ones drawn by traditional re-rankers. Second, the proportion of random recommendations in the final *RL* lists of LLM-based re-rankers (Table 6) might harm relevance, in particular for the Goodreads dataset. (As we explained in Section 4.2, when the output of the LLM contains invalid items, we replace the invalid items with randomly chosen items from *CL*, and this is likely to harm the relevance of the final *RL*.) Third, traditional re-rankers such as xQuAD and RxQuAD might have an advantage over LLM-based re-ranking as the former explicitly use user profiles to estimate relevance and genre distributions within profiles, while the latter do not have access to such information. Indeed, the LLMs might struggle to capture relevance (in terms of rank) and diversity (in terms of genres) and the interplay between the two from the prompt templates we have designed, even though the prompts explicitly highlight item relevance (by including each item's rank) and some of them (T5–T8) highlight diversity (by providing an item's genres or a description of the item) — see the detailed analysis of the prompt template results in the next subsection.

5.2 Comparison of LLM-based re-rankers

We use Figures 1 & 2 and Table 7 to draw a comparison across different prompt templates and LLM-based re-rankers for both the Anime and Goodreads datasets. In Figures 1 and 2 for Anime and Goodreads dataset respectively, each plot corresponds to a different metric. In each plot, on the horizontal axis, there are groups of results for each of the eight templates (T1–T8) and, within each group, there are three re-rankers, i.e. ChatGPT, InstructGPT, Llama2-7B-Chat. On the vertical axis, values represent the percentage difference between the re-ranker and the MF baseline. Table 7 reports the performances of the LLM-based re-rankers grouped by prompt template.

Just Accepted

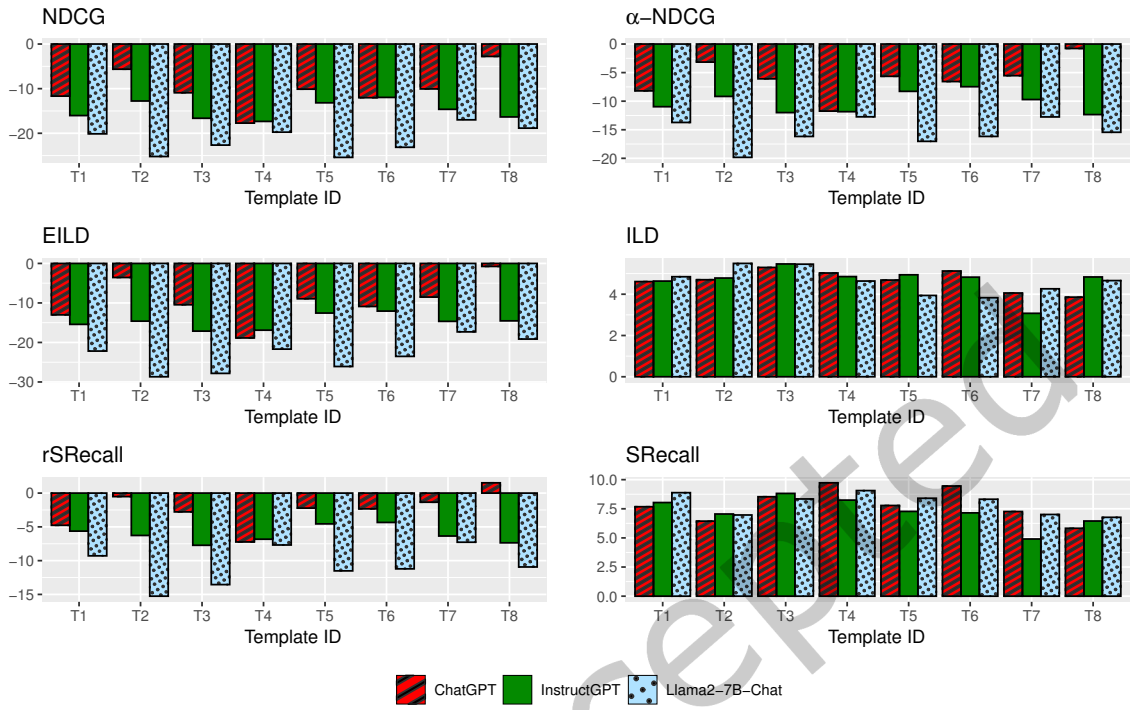

Fig. 1. Comparison of the LLM-based re-rankers across different prompt templates for the Anime dataset. For each metric, we plot the percentage difference with respect to the scores of the baseline MF. We repeat here Table 1 to facilitate interpretation of the template ids.

Template id	λ	{goal_string}	{additional_features}
T1	0.5	"balance relevance and diversity"	None
T2	0	"maximize the items' diversity in the list"	None
T3	0	"maximize the items' genre-based diversity in the list"	None
T4	0.5	"balance relevance and genre-based diversity"	None
T5	0.5	"balance relevance and diversity"	List of the candidate item's genres
T6	0.5	"balance relevance and genre-based diversity"	List of the candidate item's genres
T7	0.5	"balance relevance and diversity. Guidelines to perform the re-ranking are: Use the plot summary information of each item attached in curly bracket"	Candidate item's plot summary
T8	0	"maximize the books' diversity in the list. Guidelines to perform the re-ranking are: Use the plot summary information of each item attached in curly bracket"	Candidate item's plot summary

For ease of exposition, we do not include results for Llama2-13B-Chat (although they, along with all results are in the Appendix). Its performances is generally worse than all the other re-rankers, hence its results do not

Just Accepted

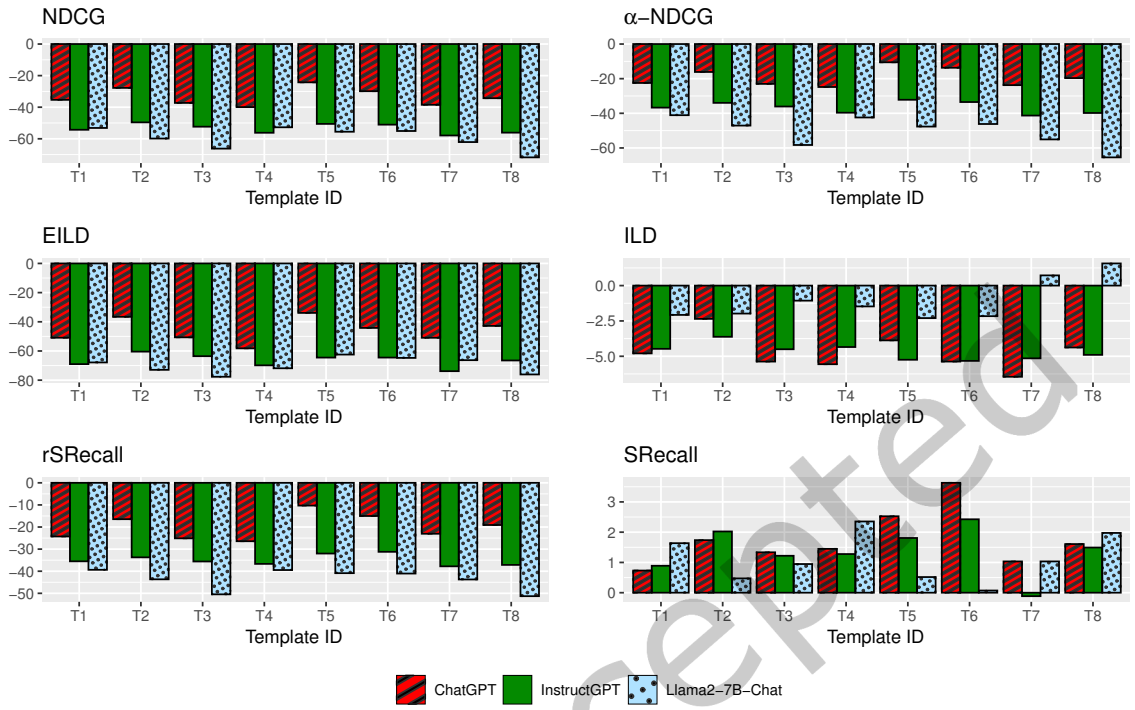

Fig. 2. Comparison of the LLM-based re-rankers across different prompt templates for the Goodreads dataset. For each metric, we plot the percentage difference with respect to the scores of the baseline MF. We repeat here Table 1 to facilitate interpretation of the template ids.

Template id	λ	{goal_string}	{additional_features}
T1	0.5	"balance relevance and diversity"	None
T2	0	"maximize the items' diversity in the list"	None
T3	0	"maximize the items' genre-based diversity in the list"	None
T4	0.5	"balance relevance and genre-based diversity"	None
T5	0.5	"balance relevance and diversity"	List of the candidate item's genres
T6	0.5	"balance relevance and genre-based diversity"	List of the candidate item's genres
T7	0.5	"balance relevance and diversity. Guidelines to perform the re-ranking are: Use the plot summary information of each item attached in curly bracket"	Candidate item's plot summary
T8	0	"maximize the books' diversity in the list. Guidelines to perform the re-ranking are: Use the plot summary information of each item attached in curly bracket"	Candidate item's plot summary

Just Accepted

add valuable insights to the discussion. It is surprising that Llama2-13B-Chat is worse than Llama2-7B-Chat, since LLM capabilities usually correlate with LLM size when considering models of the same family. However, differences in human preference alignment may play a major confounding role [46], and there are indications in the literature that Llama2-13B-Chat is worse than Llama2-13B-Chat for some tasks, e.g. see human evaluation in [57]. But, beyond these general points, we have no clear explanation for why Llama2-13B-Chat performs so poorly in our experiments and we leave its investigation for future work.

Overall, we can draw three main findings. First, LLM-based re-ranking improves the baseline ranking of MF for ILD in the Anime dataset and SRecall in both datasets (and all the LLM-based re-ranking performances except for the ones of ChatGPT are statistically significantly different from the MF ones). For the other metrics, performance degradation is more severe on the Goodreads than on the Anime dataset. The fact that the Anime dataset is less prone to invalid generation than the Goodreads dataset (2.3% vs 13.1%, Table 6) might partially explain the difference in degradation. Also, an LLM's internal knowledge about the specific recommendation domain is another key factor affecting re-ranking. We speculate that generating content related to books is more challenging than anime, probably due to the nature of the domain and its content (e.g. the presence of similar titles in a prompt or the length of the prompts, which are longer for books than for anime on average, see Table 2). However, our experiments cannot confirm our speculation or that this even applies to other domains. We plan to address this in future work.

Second, overall, ChatGPT exhibits the best performance among LLMs (within each specific template and averaged across all templates) for all metrics, with the only exception being SRecall and ILD in both datasets, where the model is not the best in all the different templates. We posit that the extent to which an LLM re-ranker draws low-rank items and includes random recommendations from CL in the final recommendation list RL correlates with performance differences across datasets and templates, which partially explain these results. Indeed, Tables 5 and 6 show that OpenAI's models (on average) draw higher ranked items from CL and need fewer random recommendations than Meta's models. This is consistent across the Anime and Goodreads datasets, but to a greater extent for the former. The only exception is between ChatGPT and InstructGPT, where the former has more random recommendations than the latter. Thus, we argue that the proportion of random recommendations only partly explains the loss of relevance in LLM-based re-rankers. We believe that a model's reasoning capability also plays a major role in performance differences. According to several benchmarks[11] and the literature [9, 32], OpenAI models are superior to Meta's models, and ChatGPT shows superior capabilities over InstructGPT (although these benchmarks and the literature are not specific to recommender systems). Our results on diversification re-ranking seem to confirm what the benchmarks and literature have shown.

Third, different prompt templates show different performance across re-rankers, datasets and metrics. This suggests that there is no best template. Prompt template design needs to be tailored to specific LLMs, recommendation domains and metrics. In the next two paragraphs, we will highlight the most significant of our findings about the different templates.

Templates T1 and T4–T7 are designed to strike a balance between relevance and diversity (similar to setting $\lambda = 0.5$) and should exhibit the best performance for relevancy-aware metrics, while T2, T3 and T8 should increase diversity the most ($\lambda = 0$). This is reflected in their average re-ranking performances, but not consistently (see Table 7). On the Anime dataset, for example, T8 and T7 are the templates where relevance-aware metrics degrade the least, even though T8 is not explicitly designed to account for relevance. Also, T2, T3 and T4 improve ILD and rSRecall the most, even though T4 should focus more on balancing relevance (but instead degrades relevance-aware metrics more than any other templates). The results of the Goodreads dataset are more consistent with what we expect from the template design. T5 and T6 show the least amount of degradation on the relevance-aware metrics as expected;T2, T8 benefit ILD the most; and T4, T6 & T8 benefit SRecall the most.

[11]https://artificialanalysis.ai/models

Just Accepted

Table 7. Average performances of ChatGPT, InstructGPT and Llama2-7B-Chat with different prompt templates. For each metric, we highlight the two best performances in bold.

	Anime						Goodreads					
	NDCG	**α-NDCG**	**EILD**	**ILD**	**rSRecall**	**SRecall**	**NDCG**	**α-NDCG**	**EILD**	**ILD**	**rSRecall**	**SRecall**
T1	-15.9	-11.0	-16.9	4.7	-6.6	8.2	-47.6	-33.4	-62.6	-3.8	-33.1	1.1
T2	-14.5	-10.7	-15.6	**5.0**	-7.3	6.8	-45.7	-32.4	**-56.6**	**-2.7**	-31.3	1.4
T3	-16.7	-11.4	-18.5	**5.4**	-8.0	**8.6**	-52.0	-39.1	-64.0	-3.6	-37.1	1.2
T4	-18.3	-12.1	-19.1	4.8	-7.2	**9.0**	-49.6	-35.6	-66.6	-3.8	-34.2	**1.7**
T5	-16.2	-10.3	-15.9	4.5	-6.1	7.8	**-43.5**	**-30.1**	**-53.6**	-3.8	**-27.7**	1.6
T6	-15.7	-10.1	-15.5	4.6	-6.0	8.3	**-45.3**	**-31.1**	-57.8	-4.3	**-29.1**	**2.0**
T7	**-13.9**	**-9.3**	**-13.5**	3.8	**-5.0**	6.4	-52.8	-40.0	-63.7	-3.6	-34.9	0.7
T8	**-12.7**	**-9.5**	**-11.5**	4.5	**-5.6**	6.3	-54.1	-41.6	-61.8	**-2.6**	-35.8	**1.7**

Table 8. Average computational time (in seconds) to re-rank the candidate recommendation of a single user in our experiments.

	Re-rankers					
	MMR	**xQuAD**	**RxQuAD**	**ChatGPT & InstructGPT**	**Llama2-7B-Chat**	**Llama2-13B-Chat**
Anime	0.05	0.1	0.122	<25	12.6	23.7
Goodreads	0.08	0.13	0.18	<25	16.1	31.6
	0.065	0.115	0.151	<25	14.35	27.6

On average, feature-aware templates (i.e. T5–T8) benefit the re-ranking performances more than templates that do not include item features (i.e. T5–T8) on both datasets, even though the feature-aware templates result in more invalid and hence random recommendations (which may affect performance negatively). Notably, T8 has the best performance on relevance-aware metrics for the Anime dataset and on diversity metrics for the Goodreads dataset despite generating the largest number of invalid recommendations (22.8%). Overall, these findings are a positive indication that LLMs can effectively use additional information to improve re-ranking results.

5.3 Re-ranking cost analysis

Finally, we analyse the cost of performing re-ranking. Table 8 reports the average inference time to re-rank the candidate recommendations of a user. For re-ranking using OpenAI's LLMs, the table shows the same value (i.e. 25 seconds). This value is not the actual cost. It is an upper bound which we have to use in this table due to problems we encountered using OpenAI APIs. In a preliminary experiment to test the APIs, we noticed that the model's response time had a very large variability and sometimes the model did not return any outputs at all (i.e. the service seemed not to be available), probably due to unknown usage policies and high workloads. We found that introducing a sleep time of 25 seconds between API calls was a way of ensuring we obtaining outputs. Unfortunately, this prevents us from providing an accurate estimation of response time. Nevertheless, the table shows that traditional re-rankers have a clear advantage over LLM-based re-rankers here: they are three orders of magnitude faster than the LLM-based re-rankers. In the case of LLM-based re-rankers, it also shows, as expected, that larger models lead to longer re-ranking time: Llama2-7B-Chat is faster than Llama2-13B-Chat.

Table 9 breaks down the financial costs in terms of API calls for the OpenAI models[12]. There are no API costs for re-ranking using Meta's LLMs since we hosted these models on our own GPUs. In principle, we could provide a cost comparison between the OpenAI and Meta model re-rankers if we were to calculate their energy costs. In practice, estimates of energy costs are hard to give due, e.g., to the volatility of energy market prices over the period during which we ran the experiments. The total cost of re-ranking for 500 users in our experiment is roughly 161$, and around 75% of this expense is incurred by InstructGPT, whose API calls are more expensive.

[12]The prices relate to the timeframe of our experiments, i.e. from October 2023 to January 2024.Prcies and pricing policies may have changed.

Just Accepted

Table 9. Total re-ranking costs using the OpenAI models for the 500 users in our experiments.

Dataset	Template id	# Input tokens		# Output tokens	
		ChatGPT	InstructGPT	ChatGPT	InstructGPT
Anime	T1-T4	~1.8M		~118k	~121k
	T5, T6	~4.2M		~159k	~210k
	T7, T8	~8.5M		~176k	~245k
Goodreads	T1-T4	~3.3M		~212k	~220k
	T5, T6	~6.9M		~221k	~285k
	T7, T8	~9.4M		~261k	~269k
	Total # of tokens	~78.4M		~1.1M	~1.3M
	Unitary price	0.5$/1M	1.5$/1M	1.5$/1M	2$/1M
	Total cost ChatGPT	78.4M · 0.5 + 1.1M · 1.5$ = **40.85$**			
	Total cost InstructGPT	78.4M · 1.5$ + 1.3M · 2$ = **120.2$**			
	Total cost	40.85$ + 120.2$ = **161.05$**			

Prices are correlated with the number of tokens processed: the Anime dataset experiment is cheaper than the one with Goodreads as the latter's item titles and features are more wordy; feature-aware prompt templates (i.e. T5-T8) are the most expensive ones (in particular T7 and T8, because item descriptions in the form of plot summaries generally have more tokens than genre lists). Therefore, recalling the results of the earlier sections, we find that ChatGPT is superior to InstructGPT in terms of accuracy/diversity and cost.

In summary, traditional re-ranking is much faster and less resource-demanding than LLM-based re-ranking. Among LLM-based re-ranking, ChatGPT is cheaper than InstructGPT, and use of these models comes with a lack of control and transparency due to the API services they rely on. Finally, Meta's models ensure more control over data and inference but need private computational resources, with difficult-to-estimate costs that are subject to market volatility. However, some recent reports[13] report that training and inference are rapidly becoming cheaper (decreasing annually by 75% and 86% respectively), which will probably break down the barrier for LLM adoption and reduce the cost advantage of traditional re-ranking.

6 Beyond performance: LLM-based and greedy re-ranking trade-offs

In the previous section we have analysed and discussed accuracy/diversity performance and costs of LLM-based re-rankers and compared them with the ones of traditional greedy re-ranking. In this section we focus on RQ4: we discuss the trade-offs RS providers should take into account when selecting one of these two re-ranking approaches. Table 10 summarizes our analysis.

Many of the traditional approaches need explicit item features to use in the re-ranking. Acquiring these features might require human labelling or other expensive information retrieval procedures. The LLM-based approaches do not need to be given explicit item features to perform the re-ranking. They can instead take advantage of implicit knowledge about items that is embedded in their parameters during pre-training or human alignment. This implicit knowledge might be more complex than the simple feature sets (e.g. genres) typically used in traditional re-ranking, and this might enhance the diversity of the final recommendations. Moreover, LLMs might even leverage cross-domain relationships; for example, when recommending movies, the LLM may implicitly leverage relationships with related books and music. On the other hand, an LLM's internal knowledge is often unknown by the users who prompt the LLM (especially for proprietary models), and it might be hard to drive the model to use its implicit knowledge when re-ranking. With prompt templates T5-T8, we attempted to reduce this

[13] https://www.ark-invest.com/big-ideas-2024

Just Accepted

Table 10. Comparison of traditional re-rankers and LLM-based re-rankers beyond re-ranking performance.

	Re-rankers	
	Traditional	**LLM-based**
Require explicit item features	Yes	No (but can use them if available)
Can re-rank large candidate lists	Yes	Depends on the size of the context window
Require expensive computations	No	Yes (but can be reduced with optimizations)
Have a heavy memory footprint	No	Yes (but can be reduced with optimizations)
Control over data, computations, usage	Full	Limited (depending on the model)
Easily address the cold-start problem	Yes	No
Always produce valid rankings	Yes	No
Easily identify items	Yes	No (depending on the domain)

gap by providing the LLM with explicit item features to use during re-ranking, similarly to what is done in RAG with LLMs [20] and in the same way we do with traditional re-rankers. The results presented in the previous section show that explicit item features can sometimes benefit re-ranking.

Greedy approaches can re-rank candidate lists composed of a high number of items (limited only by the amount of memory available), while the candidate list size when re-ranking LLM-based is limited by the size of the input context that can be fed into an LLM. Although some LLMs have long context size windows of millions of tokens (e.g. Google Gemini 1.5[14]) and some tricks are available to increase the size of such windows (e.g. [17]), the OpenAI and Meta LLMs we used all have a limited input size of roughly 4k tokens.

Most of the traditional re-ranking approaches that we presented in Section 2.2 do not require expensive computations (they can run on CPUs) and do not have high memory requirements. Moreover, simple techniques might drastically reduce the re-ranking time (for example, item distances might be pre-computed and cached). By contrast, resource requirements are one drawback of LLMs: see the insights from the previous section. Hosting and running efficiently an LLM-based service (in our scenario, an LLM-based re-ranker) needs resources such as GPUs and several gigabytes of memory. These are expensive and might require skilled technical expertise. An LLMs' high inference time might also be a bottleneck in settings where recommendations must be generated on the fly (e.g. session-based recommenders). There is a growing amount of recent work on both reducing LLM inference time (e.g. using batch inference, flash attention, and weight sharing) and reducing their memory footprint (e.g. using quantization and model distillation) (see [69] for a broad overview). However, the gap with traditional re-rankers remains substantial.

When using traditional re-rankers, data and computations are typically hosted on a local infrastructure, as the cost and the technical challenges are quite low. By contrast, when running LLM-based re-ranking using a proprietary model or a proprietary platform (as we do with ChatGPT leveraging OpenAI APIs, see Section 4), control over data and processing is generally limited. For example, data needs to be uploaded to and downloaded from the data centre where the LLM is hosted; the inference time (i.e. generation time) might be higher when the endpoint is overloaded; inference availability may even be disrupted; usage limits on the service might be applied; and there is need for the external platform provider to guarantee data protection. However, when using open source alternatives such as Meta's LLMs, these problems are mitigated because control over data and computations is retained.

From a re-ranking point of view, traditional approaches address the well-known item cold-start problem fairly smoothly: they require only that each new item be associated with its features. By contrast, LLM-based approaches might struggle with this problem. Items that are new to the system (e.g. newly released movies) and

[14]https://deepmind.google/technologies/gemini/

Just Accepted

information related to them (such as item features or general descriptions) will be unknown to an LLM that was trained or tuned prior to release of the new item. Updating its parameters might be impractical even if fresh item information is available. Re-training will likely be periodic, possibly introducing a substantial delay before the LLM becomes aware of the new items. One solution to this problem might be the RAG approach [20], which could be used to gather fresh contextual information related to items to enrich the re-ranking prompts. However, implementing a RAG system could be complex and expensive and would also increase the re-ranking time due to the inference time of the retrieval components.

Traditional re-rankers always produce valid rankings. By contrast, LLMs may generate text that is inconsistent, hallucinated, harmful, or irrelevant to the instructions provided in the prompt. The results of our experiments when using LLM-based re-rankers also report invalid (malformed) outputs, e.g. recommendations that have titles of items that do not exist, or titles that do not appear in *CL* (see Section 5). Countermeasures have been proposed in the literature to mitigate such problems [30, 74], but it appears there is no one-size-fits-all solution. Advanced parsing of an LLM's output can also be implemented (and we indeed partly address invalid generations with this technique in Section 4.2), but these solutions do not necessarily scale well to a wide range of invalid generation corner cases.

Traditional re-rankers typically identify items by identifiers such as product codes. This can work across different recommendation domains. In LLM-based re-ranking, the prompts we use rely on working in domains where items have useful titles to identify them. This allows the LLM to draw on its knowledge of these items. Domains such as books, movies, and music use titles in this way but this might be challenging or impractical in other domains. For example, consider domains where people are recommended to other people, such as dating or social media. An LLM may not know who the individuals are by their names. If they do know, it will be because the training data includes data about these individuals (and whether this is so is often unknown upfront, as training data is often undisclosed for state-of-the-art models). Non-uniqueness may also be a problem (the extent of the problem perhaps being greater in some domains, such as hotels, than others, such as movies). Names and titles may need to be extended (e.g. hotels with their cities) to reduce non-uniqueness. Finally, there are domains where there may not be titles at all, e.g. short posts to social media streams. When titles/names are unavailable, LLMs might still use items features or descriptions to identify items and re-rank them with proper prompts modification. In our future work we plan to explore re-ranking in this challenging scenario.

7 Conclusion and future work

This paper is a first step in investigating the use of general-purpose conversational LLMs to enhance top-n recommendation diversity through re-ranking. In this paper, we proposed to prompt LLMs in a zero-shot fashion to re-rank a large set of candidate items and provide a final recommendation set. We wanted to discover whether LLMs can interpret diversification re-ranking. We also explored the impact of different prompt templates we designed: some of them aim to strike a balance between relevance and diversity, and some focus on improving diversity only. Some templates include item features as contextual information to help re-ranking, and others do not. We run experiments on two publicly available datasets (from the anime movies and books domains) where we compare LLM-based re-ranking such as ChatGPT and InstructGPT from OpenAI and Llama2-7B-Chat and Llama2-13B-Chat from Meta with traditional greedy diversification re-rankers and a random re-ranker. We analyzed their cost-performance trade-offs and provided a comprehensive discussion of the pros and cons beyond accuracy/diversity performances and computational costs to help inform the choice between LLM-based and traditional greedy approaches to perform diversification re-ranking.

First of all, our results show that the LLM-based re-rankers that we have proposed can interpret the re-ranking task. For both datasets, all re-rankers sacrifice relevance to improve diversity, but to a different extent. LLM-based rankings have better relevance/diversity trade-offs than the ones of random re-ranking but worse than the ones of

Just Accepted

the traditional re-rankers. We argue that LLM-based re-rankers are inferior to traditional re-rankers, particularly for relevance-aware metrics, because: they draw lower-ranked items from the candidate list; they are penalised by the presence of random recommendations in the final list due to invalid generation; and they sometimes struggle to effectively capture relevance and item diversity, whereas traditional re-rankers directly leverage user profiles and item features to drive re-ranking.

Re-ranking from OpenAI models are better than the ones from Meta's models: ChatGPT is, in general, the best performing LLM re-ranker, and Llama2-13B-Chat is the worst (although comparable to the others on ILD and SRecall). Our findings are in line with the majority of the literature that credits OpenAI models with more advanced capabilities than Llama models. Also, different prompt templates have different performances across LLMs and datasets, which suggests that their design should be tailored accordingly. Even so some patterns can be discerned. For example, on average, feature-aware templates benefit the re-ranking performances more than templates that do not include item features; and templates that are designed to boost diversity rather than striking a balance between accuracy and diversity do not consistently achieve this aim.

Finally, traditional re-ranking is much faster and less resource-demanding than LLM-based re-ranking. When using LLM-based re-ranking, one should carefully consider the costs for inference and deployment and constraints on the LLM usage (such as control over data and inference rate/time).

There are many directions for future work to extend the study reported in this paper. LLMs are improving constantly: models are getting larger, maximum context sizes are growing, and inference costs are falling. It would be valuable to repeat our study in the future. We might expect the gap between LLM-based re-ranking and traditional re-ranking to fall. With reduced costs, it would also become feasible to run experiments with more test users, a wider range of prompts, and more than two datasets.

There are also several ways to improve LLM-based re-rankers to see whether their performance can come closer to that of traditional re-rankers. One option is to train a dedicated LLM to perform the re-ranking. This can be done, for example, by combining SFT and RLHF with proper instruction/response pairs and human feedback or by deriving a distilled model [23]. Another option is to reduce the proportion of invalid items (and hence random recommendations) by repeating the generation or implementing more advanced inexact string matching. This might help the quality of recommendations, particularly their relevance. Improving the prompts with richer contextual information might also help. This can be done in several ways: by providing demonstrations of re-ranking in the prompts (in the spirit of few-shot learning); implementing a RAG pipeline to enrich item features; or by making the re-ranking instructions in the prompts more detailed and explicit to break down the task into individual steps (similarly to what is done in Chain-of-thoughts prompting [67]), e.g. `Generate a new ranking where, at each step, you select an item that is different from the ones already included in the final recommendation`.

LLMs are essentially black-box models. There are only a few attempts to gain insight into what they learn (e.g. [2]). Research into explaining how LLMs perform diversification re-ranking could be another direction for future work.

Finally, re-ranking quality could be assessed with alternative diversity metrics (e.g. measuring item distances on item ratings) or other beyond-accuracy metrics such as novelty and serendipity [31] to explore LLM capabilities more fully.

Acknowledgments

This publication has emanated from research supported by Science Foundation Ireland under Grant No. 12/RC/2289-P2 at the Insight SFI Research Centre for Data Analytics at UCC, which is co-funded under the European Regional Development Fund.

Just Accepted

References

[1] Rakesh Agrawal, Sreenivas Gollapudi, Alan Halverson, and Samuel Ieong. 2009. Diversifying search results. In *Proceedings of the second ACM international conference on web search and data mining*. 5–14.

[2] Anthropic. 2024. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html.

[3] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys '23)*. Association for Computing Machinery, New York, NY, USA, 1007–1014. https://doi.org/10.1145/3604915.3608857

[4] Vadim Borisov, Kathrin Seßler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. 2022. Language models are realistic tabular data generators. *arXiv preprint arXiv:2210.06280* (2022).

[5] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*. 335–336.

[6] Diego Carraro. 2020. *Active learning in recommender systems: an unbiased and beyond-accuracy perspective*. Ph. D. Dissertation. University College Cork.

[7] Pablo Castells, Neil Hurley, and Saul Vargas. 2021. Novelty and diversity in recommender systems. In *Recommender systems handbook*. Springer, 603–646.

[8] Hanxiong Chen, Xu Chen, Shaoyun Shi, and Yongfeng Zhang. 2021. Generate natural language explanations for recommendation. *arXiv preprint arXiv:2101.03392* (2021).

[9] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E Gonzalez, et al. 2024. Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132* (2024).

[10] Charles LA Clarke, Maheedhar Kolla, Gordon V Cormack, Olga Vechtomova, Azin Ashkan, Stefan Büttcher, and Ian MacKinnon. 2008. Novelty and diversity in information retrieval evaluation. In *Proceedings of the 31st annual international ACM SIGIR conference on Research and development in information retrieval*. 659–666.

[11] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. M6-rec: Generative pretrained language models are open-ended recommender systems. *arXiv preprint arXiv:2205.08084* (2022).

[12] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering ChatGPT's Capabilities in Recommender Systems. *arXiv preprint arXiv:2305.02182* (2023).

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[14] Tommaso Di Noia, Jessica Rosati, Paolo Tomeo, and Eugenio Di Sciascio. 2017. Adaptive multi-attribute diversity for recommender systems. *Information Sciences* 382 (2017), 234–253.

[15] Dario Di Palma. 2023. Retrieval-augmented Recommender System: Enhancing Recommender Systems with Large Language Models. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) *(RecSys '23)*. Association for Computing Machinery, New York, NY, USA, 1369–1373. https://doi.org/10.1145/3604915.3608889

[16] Hao Ding, Yifei Ma, Anoop Deoras, Yuyang Wang, and Hao Wang. 2021. Zero-shot recommender systems. *arXiv preprint arXiv:2105.08318* (2021).

[17] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. 2024. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. *arXiv preprint arXiv:2402.13753* (2024).

[18] Michael D Ekstrand, F Maxwell Harper, Martijn C Willemsen, and Joseph A Konstan. 2014. User perception of differences in recommender algorithms. In *Proceedings of the 8th ACM Conference on Recommender systems*. 161–168.

[19] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).

[20] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL]

[21] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*. 299–315.

[22] Sander Greenland, Stephen J Senn, Kenneth J Rothman, John B Carlin, Charles Poole, Steven N Goodman, and Douglas G Altman. 2016. Statistical tests, P values, confidence intervals, and power: a guide to misinterpretations. *European journal of epidemiology* 31, 4 (2016), 337–350.

[23] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. Knowledge Distillation of Large Language Models. arXiv:2306.08543 [cs.CL]

Just Accepted

[24] Deepesh V. Hada, Vijaikumar M., and Shirish K. Shevade. 2021. ReXPlug: Explainable Recommendation Using Plug-and-Play Language Model. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*. Association for Computing Machinery, New York, NY, USA, 81–91. https://doi.org/10.1145/3404835.3462939

[25] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning vector-quantized item representation for transferable sequential recommenders. In *Proceedings of the ACM Web Conference 2023*. 1162–1171.

[26] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 585–593.

[27] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large Language Models are Zero-Shot Rankers for Recommender Systems. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part II*. Springer-Verlag, Berlin, Heidelberg, 364–381. https://doi.org/10.1007/978-3-031-56060-6_24

[28] Neil J Hurley. 2013. Personalised ranking with diversity. In *Proceedings of the 7th ACM Conference on Recommender Systems*. 379–382.

[29] Tamas Jambor and Jun Wang. 2010. Optimizing multiple objectives in collaborative filtering. In *Proceedings of the fourth ACM conference on Recommender systems*. 55–62.

[30] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.

[31] Marius Kaminskas and Derek Bridge. 2016. Diversity, serendipity, novelty, and coverage: a survey and empirical analysis of beyond-accuracy objectives in recommender systems. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 7, 1 (2016), 1–42.

[32] Navreet Kaur, Monojit Choudhury, and Danish Pruthi. 2023. Evaluating Large Language Models for Health-related Queries with Presuppositions. *arXiv preprint arXiv:2312.08800* (2023).

[33] Mesut Kaya and Derek Bridge. 2019. Subprofile-aware diversification of recommendations. *User Modeling and User-Adapted Interaction* 29 (2019), 661–700.

[34] John Paul Kelly and Derek Bridge. 2006. Enhancing the diversity of conversational collaborative recommendations: a comparison. *Artificial Intelligence Review* 25 (2006), 79–95.

[35] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.

[36] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian McAuley. 2023. Text Is All You Need: Learning Language Representations for Sequential Recommendation. *arXiv preprint arXiv:2305.13731* (2023).

[37] Jinming Li, Wentao Zhang, Tian Wang, Guanglei Xiong, Alan Lu, and Gerard Medioni. 2023. GPT4Rec: A generative framework for personalized recommendation and user interests interpretation. *arXiv preprint arXiv:2304.03879* (2023).

[38] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Personalized prompt learning for explainable recommendation. *ACM Transactions on Information Systems* 41, 4 (2023), 1–26.

[39] Xinyi Li, Yongfeng Zhang, and Edward C Malthouse. 2023. A Preliminary Study of ChatGPT on News Recommendation: Personalization, Provider Fairness, Fake News. *arXiv preprint arXiv:2306.10702* (2023).

[40] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. *arXiv preprint arXiv:2306.05817* (2023).

[41] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.

[42] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2023. A First Look at LLM-Powered Generative News Recommendation. *arXiv preprint arXiv:2305.06566* (2023).

[43] Yiding Liu, Weixue Lu, Suqi Cheng, Daiting Shi, Shuaiqiang Wang, Zhicong Cheng, and Dawei Yin. 2021. Pre-trained language model for web-scale retrieval in baidu search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3365–3375.

[44] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *Comput. Surveys* 56, 2 (2023), 1–40.

[45] Sheshera Mysore, Andrew McCallum, and Hamed Zamani. 2023. Large Language Model Augmented Narrative Driven Recommendations. *arXiv preprint arXiv:2306.02250* (2023).

[46] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[47] Aleksandr V Petrov and Craig Macdonald. 2023. Generative Sequential Recommendation with GPTRec. *arXiv preprint arXiv:2306.11114* (2023).

Just Accepted

[48] István Pilászy, Dávid Zibriczky, and Domonkos Tikk. 2010. Fast als-based matrix factorization for explicit and implicit feedback datasets. In *Proceedings of the fourth ACM conference on Recommender systems*. 71–78.
[49] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
[50] Sajjad Rahmani, AmirHossein Naghshzan, and Latifa Guerrouj. 2023. Improving Code Example Recommendations on Informal Documentation Using BERT and Query-Aware LSH: A Comparative Study. *arXiv preprint arXiv:2305.03017* (2023).
[51] Marco Tulio Ribeiro, Anisio Lacerda, Adriano Veloso, and Nivio Ziviani. 2012. Pareto-efficient hybridization for multi-objective recommender systems. In *Proceedings of the sixth ACM conference on Recommender systems*. 19–26.
[52] Yue Shi, Xiaoxue Zhao, Jun Wang, Martha Larson, and Alan Hanjalic. 2012. Adaptive diversification of recommendation results via latent factor portfolio. In *Proceedings of the 35th international ACM SIGIR conference on Research and development in information retrieval*. 175–184.
[53] Barry Smyth and Paul McClave. 2001. Similarity vs. diversity. In *International conference on case-based reasoning*. Springer, 347–361.
[54] Ruilong Su, Li'Ang Yin, Kailong Chen, and Yong Yu. 2013. Set-oriented personalized ranking for diversified top-n recommendation. In *Proceedings of the 7th ACM Conference on Recommender Systems*. 415–418.
[55] Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agent. *arXiv preprint arXiv:2304.09542* (2023).
[56] Mirac Suzgun, Luke Melas-Kyriazi, and Dan Jurafsky. 2022. Prompt-and-rerank: A method for zero-shot and few-shot arbitrary textual style transfer with small language models. *arXiv preprint arXiv:2205.11503* (2022).
[57] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
[58] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. 2023. On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems* 36 (2023), 75993–76005.
[59] Saúl Vargas. 2015. *Novelty and diversity evaluation and enhancement in recommender systems*. Ph. D. Dissertation. PhD thesis, Universidad Autónoma de Madrid, Spain.
[60] Saúl Vargas, Linas Baltrunas, Alexandros Karatzoglou, and Pablo Castells. 2014. Coverage, redundancy and size-awareness in genre diversity for recommender systems. In *Proceedings of the 8th ACM Conference on Recommender systems*. 209–216.
[61] Saúl Vargas and Pablo Castells. 2011. Rank and relevance in novelty and diversity metrics for recommender systems. In *Proceedings of the fifth ACM conference on Recommender systems*. 109–116.
[62] Saul Vargas, Pablo Castells, and David Vallet. 2011. Intent-oriented diversity in recommender systems. In *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*. 1211–1212.
[63] Saúl Vargas, Pablo Castells, and David Vallet. 2012. Explicit relevance models in intent-oriented information retrieval diversification. In *Proceedings of the 35th international ACM SIGIR conference on Research and development in information retrieval*. 75–84.
[64] Lei Wang and Ee-Peng Lim. 2023. Zero-Shot Next-Item Recommendation using Large Pretrained Language Models. *arXiv preprint arXiv:2304.03153* (2023).
[65] Yifan Wang, Weizhi Ma, Min Zhang, Yiqun Liu, and Shaoping Ma. 2023. A survey on the fairness of recommender systems. *ACM Transactions on Information Systems* 41, 3 (2023), 1–43.
[66] William Webber, Alistair Moffat, and Justin Zobel. 2010. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems (TOIS)* 28, 4 (2010), 1–38.
[67] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
[68] Mark Wilhelm, Ajith Ramanathan, Alexander Bonomo, Sagar Jain, Ed H Chi, and Jennifer Gillenwater. 2018. Practical diversified recommendations on youtube with determinantal point processes. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. 2165–2173.
[69] Canwen Xu and Julian McAuley. 2023. A survey on model compression and acceleration for pretrained language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 10566–10575.
[70] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems* 36 (2024).
[71] Mi Zhang and Neil Hurley. 2008. Avoiding monotony: improving the diversity of recommendation lists. In *Proceedings of the 2008 ACM conference on Recommender systems*. 123–130.
[72] Qi Zhang, Jingjie Li, Qinglin Jia, Chuyuan Wang, Jieming Zhu, Zhaowei Wang, and Xiuqiang He. 2021. UNBERT: User-News Matching BERT for News Recommendation.. In *IJCAI*. 3356–3362.
[73] Xinyang Zhang, Yury Malkov, Omar Florez, Serim Park, Brian McWilliams, Jiawei Han, and Ahmed El-Kishky. 2022. TwHIN-BERT: a socially-enriched pre-trained language model for multilingual Tweet representations. *arXiv preprint arXiv:2209.07562* (2022).
[74] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *arXiv preprint arXiv:2309.01219* (2023).

Just Accepted

[75] Zizhuo Zhang and Bang Wang. 2023. Prompt learning for news recommendation. *arXiv preprint arXiv:2304.05263* (2023).
[76] Cai-Nicolas Ziegler, Sean M McNee, Joseph A Konstan, and Georg Lausen. 2005. Improving recommendation lists through topic diversification. In *Proceedings of the 14th international conference on World Wide Web*. 22–32.
[77] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593* (2019).

Just Accepted

A Examples of Prompts and outputs

A.1 Preliminary study

Q1-P1. This prompt assesses whether ChatGPT can re-rank a list of anime titles into alphabetical order.

```
You are given a list of 20 anime, delimited by triple backticks.
Your task is to re-rank this list in alphabetical order.
```
```
0.-> Hundred
1.-> The Millionaire Detective - Balance:UNLIMITED
2.-> Freaky Highschool
3.-> Arte
4.-> Infinite Ryvius
5.-> Is this A Zombie? of the Dead
6.-> A Christmas Adventure
7.-> Junjo Romantica
8.-> Rokka:Braves of the Six Flowers
9.-> Wooser's Hand-to-Mouth Life:Awakening Arc
10.-> Jormungand Season 2:Perfect Order
11.-> Genshiken
12.-> Shirobako
13.-> Scum's Wish
14.-> Inuyashiki:Last Hero
15.-> My Name Is Teppei
16.-> Magical Project S
17.-> Hand Shakers
18.-> Tekkaman Blade
19.-> Pretty Cure
```

ChatGPT's response to Q1-P1

```
```
6.-> A Christmas Adventure
3.-> Arte
2.-> Freaky Highschool
11.-> Genshiken
17.-> Hand Shakers
0.-> Hundred
14.-> Inuyashiki:Last Hero
4.-> Infinite Ryvius
5.-> Is this A Zombie? of the Dead
10.-> Jormungand Season 2:Perfect Order
7.-> Junjo Romantica
15.-> My Name Is Teppei
19.-> Pretty Cure
8.-> Rokka:Braves of the Six Flowers
```

Just Accepted
```

```
13.-> Scum's Wish
12.-> Shirobako
16.-> Magical Project S
18.-> Tekkaman Blade
1.-> The Millionaire Detective - Balance:UNLIMITED
9.-> Wooser's Hand-to-Mouth Life:Awakening Arc
```

Only "Magical Project S" is out of position.

Q1-P2. This prompt assesses whether ChatGPT can re-rank a list of anime titles by popularity.

```
You are given a list of 20 anime, delimited by triple backticks.
Your task is to re-rank the anime in this list by popularity.
```
```
Tsubasa RESERVoir CHRoNiCLE Season Two
El-Hazard:The Wanderers
Gintama Season 2
Beet the Vandel Buster
Umamusume:Pretty Derby
Bobobo-bo Bo-bobo
GG Bond:Ultimate Battle Prequel
Rozen Maiden:Zuruckspulen
Cardfight!! Vanguard:Shinemon Arc
Prince Mackaroo
Samurai Deeper Kyo
New Game!!
Trigun
ChaoS;Child
Beautiful Bones -Sakurako's Investigation-
The Ones Within
Tiger & Bunny
Cat Eyed Boy
Vampire Princess Miyu
Show By Rock!! 2
```

ChatGPT's response to Q1-P2

```
Based on popularity, the ranked list of anime would be:

```
1. Gintama Season 2
2. Trigun
3. New Game!!
4. Tiger & Bunny
5. Tsubasa RESERVoir CHRoNiCLE Season Two
6. Umamusume:Pretty Derby
```

Just Accepted
```

```
7. Rozen Maiden:Zuruckspulen
8. ChaoS;Child
9. Beautiful Bones -Sakurako's Investigation-
10. Vampire Princess Miyu
11. Samurai Deeper Kyo
12. Bobobo-bo Bo-bobo
13. Cardfight!! Vanguard:Shinemon Arc
14. The Ones Within
15. El-Hazard: The Wanderers
16. Beet the Vandel Buster
17. Show By Rock!! 2
18. GG Bond: Ultimate Battle Prequel
19. Prince Mackaroo
20. Cat Eyed Boy
```

We extracted the ground-truth popularity of these 20 anime movies from the dataset presented in Section 4. The ranking proposed by ChatGPT overlaps with the ground-truth ranking by a 0.79 score according to the rank-biased-overlap metric [66]. We ran several examples (with different item lists to re-rank), and the rank-biased-overlap scores vary between 0.5 and 0.8.

Q1-P3. This prompt assesses whether ChatGPT can re-rank a list of anime titles by release date.

```
You are given a list of 20 anime, delimited by triple backticks.
Your task is to re-rank anime in this list by release date (from
newest to oldest).
```
```
Digimon Adventure 02
Chobits
Fighting Spirit
Chrono Crusade
The Great Passage
Kiba
Green Green
Poccolies
Sekko Boys
Teasing Master Takagi-san
Steel Angel Kurumi
A Centaur's Life
Persona:Trinity Soul
Magical Princess Minky Momo:Hold on to Your Dreams
Coral Investigates
The Pet Girl of Sakurasou
The Affectuous Family
Earl and Fairy
Vividred Operation
Golden Time
```

Just Accepted

```
```

ChatGPT's response to Q1-P3

```
Here is the list of anime ranked by release date from newest to
oldest:

1. Poccolies
2. The Great Passage
3. Coral Investigates
4. Teasing Master Takagi-san
5. The Pet Girl of Sakurasou
6. Vividred Operation
7. Golden Time
8. A Centaur's Life
9. Persona: Trinity Soul
10. The Affectuous Family
11. Chobits
12. Steel Angel Kurumi
13. Chrono Crusade
14. Fighting Spirit
15. Kiba
16. Green Green
17. Digimon Adventure 02
18. Magical Princess Minky Momo: Hold on to Your Dreams
19. Earl and Fairy
20. Sekko Boys
```

We extracted the ground-truth item release dates from the dataset presented in Section 4. The ranking proposed by ChatGPT overlaps with the ground-truth ranking by a 0.79 score according to the rank-biased-overlap metric. We ran several examples (with different item lists to re-rank), and the rank-biased-overlap scores vary between 0.35 and 0.7.

Q2-P1. This prompt assesses whether ChatGPT can select the more diverse of two lists of anime titles.

```
You are given two lists of 10 anime each, list A and list B,
delimited by triple backticks.
Which of the two lists is more diverse?

```
LIST A:
Mr. Stain on Junk Alley
Let's Dance with Papa
King of Classroom, Yamazaki
Poo-Neko
The Adventures of the Little Koala
Oscar's Oasis
The Affectuous Family
```

Just Accepted
```

```
Little Wansa
Hippo and Thomas
Weekly Shimako

LIST B:
Hundred
The Millionaire Detective - Balance:UNLIMITED
Freaky Highschool
Arte
Infinite Ryvius
Is this A Zombie? of the Dead
A Christmas Adventure
Junjo Romantica
Rokka:Braves of the Six Flowers
Wooser's Hand-to-Mouth Life:Awakening Arc
Jormungand Season 2:Perfect Order
```

ChatGPT's response to Q2-P1

```
Diversity in anime can be assessed based on various factors such
as genre, themes, art style, target audience, and more.
Let's analyze the two lists in terms of genre diversity since
it's a common way to measure variety in anime.

List A:

    Mr. Stain on Junk Alley: Stop-motion animation, comedy
    Let's Dance with Papa: Slice of life, comedy
    King of Classroom, Yamazaki: Comedy, school
    Poo-Neko: Adventure, fantasy
    The Adventures of the Little Koala: Adventure, children
    Oscar's Oasis: Adventure, comedy
    The Affectionate Family: Comedy, family
    Little Wansa: Adventure, children
    Hippo and Thomas: Adventure, children
    Weekly Shimako: Slice of life, comedy

List B:

    Hundred: Action, harem
    The Millionaire Detective - Balance:UNLIMITED: Mystery, drama
    Freaky Highschool: Supernatural, comedy
    Arte: Historical, drama
    Infinite Ryvius: Sci-fi, drama
    Is this A Zombie? of the Dead: Action, comedy, supernatural
```

Just Accepted

```
    A Christmas Adventure: Christmas, adventure
    Junjo Romantica: Romance, drama
    Rokka: Braves of the Six Flowers: Fantasy, mystery
    Wooser's Hand-to-Mouth Life: Awakening Arc: Comedy, slice of
    life
    Jormungand Season 2: Perfect Order: Action, adventure

In terms of genre diversity, both lists showcase a variety of
genres. However, List B appears to have a broader range of
genres, including action, mystery, supernatural, historical,
romance, and Christmas-themed anime. List A, on the other hand,
leans more towards adventure, comedy, and slice of life genres.
Therefore, List B is more diverse in terms of genres.
```

The answer is "correct" and consistent with a genre-based diversity perspective because the set of genres that ChatGPT associates with the movies in list B is larger than the set of genres it associates with the movies in list A. This is also "correct" if we consider the ground-truth (the genres in the Anime dataset): in the dataset, the movies in list B are assigned a handful of genres, whereas the movies in list A all have one genre only ("comedy").

Q2-P2. This prompt assesses whether ChatGPT can select the more diverse of two lists of anime titles.

```
You are given two lists of 10 anime each, list A and list B,
delimited by triple backticks.
Which of the two lists is more diverse?

```
LIST A:
Ladies versus Butlers!
La storia della Arcana Famiglia
Sirius the Jaeger
School Babysitters
Pop Team Epic
Servant x Service
Uta no Prince Sama
Dragonar Academy
NAKAIMO - My Little Sister Is Among Them!
His and Her Circumstances

LIST B:
Jormungand Season 2:Perfect Order
Genshiken
Shirobako
Scum's Wish
Inuyashiki:Last Hero
My Name Is Teppei
Magical Project S
Hand Shakers
```

Just Accepted
```

```
Tekkaman Blade
Pretty Cure
```

ChatGPT's response to Q2-P2

```
To determine which list is more diverse, we can consider the
variety of genres, themes, and settings present in each list.
It's important to note that diversity can be subjective, and
different criteria can be used to evaluate it. Here, I'll
consider the diversity based on the variety of genres and themes.

List A:
1. Ladies versus Butlers! - Ecchi, Comedy, Romance
2. La storia della Arcana Famiglia - Action, Mystery, Romance,
Supernatural
3. Sirius the Jaeger - Action, Supernatural, Historical, Vampire
4. School Babysitters - Slice of Life, Comedy
5. Pop Team Epic - Comedy, Parody
6. Servant x Service - Slice of Life, Comedy
7. Uta no Prince Sama - Music, Romance
8. Dragonar Academy - Action, Fantasy, Romance
9. NAKAIMO - My Little Sister Is Among Them! - Harem, Comedy,
Romance
10. His and Her Circumstances - Romance, Drama, Comedy, School

List B:
1. Jormungand Season 2: Perfect Order - Action, Adventure
2. Genshiken - Slice of Life, Comedy
3. Shirobako - Slice of Life, Comedy, Drama
4. Scum's Wish - Drama, Romance
5. Inuyashiki: Last Hero - Sci-Fi, Action, Drama
6. My Name Is Teppei - Comedy, Sports
7. Magical Project S - Magic, Comedy, Fantasy
8. Hand Shakers - Action, Supernatural
9. Tekkaman Blade - Action, Adventure, Mecha
10. Pretty Cure - Mahou Shoujo, Magic, Fantasy

Observations:
- List A covers a variety of genres, including ecchi, action,
slice of life, comedy, romance, and supernatural.
- List B also offers a mix of genres, including action, slice
of life, comedy, drama, sci-fi, sports, magic, and mecha.

Both lists display diversity, but List A may have a slightly
broader range of genres. It includes ecchi, which is not
```

Just Accepted

```
represented in List B. However, diversity can be subjective,
and different viewers may have different preferences for what
they consider diverse.
```

Again, the answer is "correct", and for the same reasons as for Q2-P2.

Q2-P3. This prompt assesses whether ChatGPT can select the more diverse of two lists of anime titles.

```
You are given two lists of 10 anime each, list A and list B,
delimited by triple backticks.
Which of the two lists is more diverse?

```
LIST A:

The Millionaire Detective - Balance:UNLIMITED
Freaky Highschool
Arte
Infinite Ryvius
Is this A Zombie? of the Dead
A Christmas Adventure
Junjo Romantica
Rokka:Braves of the Six Flowers
Wooser's Hand-to-Mouth Life:Awakening Arc

LIST B:
Jormungand Season 2:Perfect Order
Genshiken
Shirobako
Scum's Wish
Inuyashiki:Last Hero
My Name Is Teppei
Magical Project S
Hand Shakers
Tekkaman Blade
Pretty Cure
```
```

ChatGPT's response to Q2-P3

```
To assess diversity in anime, we can consider various factors such
as genre, themes, art style, and target audience.
Let's analyze the two lists in terms of genre diversity:

List A:

    The Millionaire Detective - Balance:UNLIMITED: Mystery, drama
    Freaky Highschool: Supernatural, comedy
```

Just Accepted

```
    Arte: Historical, drama
    Infinite Ryvius: Sci-fi, drama
    Is this A Zombie? of the Dead: Action, comedy, supernatural
    A Christmas Adventure: Christmas, adventure
    Junjo Romantica: Romance, drama
    Rokka: Braves of the Six Flowers: Fantasy, mystery
    Wooser's Hand-to-Mouth Life: Awakening Arc: Comedy, slice of
    life

List B:

    Jormungand Season 2: Perfect Order: Action, adventure
    Genshiken: Comedy, slice of life
    Shirobako: Slice of life, comedy
    Scum's Wish: Drama, romance
    Inuyashiki: Last Hero: Action, drama, sci-fi
    My Name Is Teppei: Sports, drama
    Magical Project S: Magic, comedy
    Hand Shakers: Action, drama, fantasy
    Tekkaman Blade: Action, sci-fi
    Pretty Cure: Action, adventure, magic

In terms of genre diversity, both lists showcase a variety of
genres. List B, however, seems to have a broader range,
including action, adventure, comedy, slice of life, drama,
romance, sci-fi, magic, and sports. List A also covers multiple
genres, but it has a stronger emphasis on drama and romance,
with fewer distinct genres represented. Therefore, List B is more
diverse in terms of genres.
```

This time the answer is "incorrect" as list A is more diverse than list B, according both to the genres that ChatGPT associates with the two lists and the genres extracted from the Anime dataset.

A.2 Diversification re-ranking

Example of a prompt from template T1 and using the anime dataset.

```
You are given a ranked recommendation list of 40 items for a user,
delimited by triple backticks.
Your task is to re-rank this candidate list and provide a final
top-10 recommendation list where the goal is to balance relevance
and diversity. Strictly use the following format for the output,
and don't provide additional  information.

1-> <anime name>
2-> <anime name>
...
```


Just Accepted

```
10-> <anime name>

```
1. Dragon Ball Z
2. Naruto
3. Deadman Wonderland
...
9. Code Geass:Lelouch of the Rebellion R2
10. One Punch Man
...
15. Haikyu!!
...
22. Made in Abyss
...
27. Death Parade
...
30. Attack on Titan Season 3 Part 2
...
32. Re:ZERO -Starting Life in Another World
...
34. The Promised Neverland
35. Demon Slayer:Kimetsu no Yaiba
...
40. Your Lie in April
```
```

ChatGPT's output for the previous prompt.

```
1-> Deadman Wonderland
2-> Code Geass:Lelouch of the Rebellion R2
3-> One Punch Man
4-> Death Parade
5-> Attack on Titan Season 3 Part 2
6-> Re:ZERO -Starting Life in Another World
7-> Made in Abyss
8-> Haikyu!!
9-> The Promised Neverland
10-> Demon Slayer:Kimetsu no Yaiba
```

B Results

Just Accepted

Table 11. Re-ranking performance for the Anime dataset. All metrics are computed with a cutoff of size $n = |RL| = 10$. For all metrics, the higher the scores, the better the performance. For each measure, we also report the confidence intervals calculated with a 95% confidence level.

		NDCG	**α-NDCG**	**EILD**	**ILD**	**rSRecall**	**SRecall**
MF		0.316 ± 0.020	0.325 ± 0.019	0.221 ± 0.015	0.776 ± 0.006	0.234 ± 0.012	0.473 ± 0.006
Re-ranker	**Template**						
Random		0.130 ± 0.012	0.154 ± 0.013	0.094 ± 0.011	0.811 ± 0.004	0.145 ± 0.010	0.499 ± 0.005
MMR		0.299 ± 0.019	0.313 ± 0.018	0.222 ± 0.015	0.874 ± 0.003	0.237 ± 0.013	0.544 ± 0.005
xQuad		0.301 ± 0.019	0.350 ± 0.021	0.212 ± 0.015	0.780 ± 0.005	0.251 ± 0.013	0.539 ± 0.006
RxQuad		0.295 ± 0.019	0.338 ± 0.020	0.207 ± 0.014	0.770 ± 0.005	0.244 ± 0.013	0.506 ± 0.006
ChatGPT	T1	0.279 ± 0.018	0.298 ± 0.018	0.192 ± 0.014	0.812 ± 0.005	0.223 ± 0.012	0.509 ± 0.005
	T2	0.298 ± 0.019	0.314 ± 0.018	0.213 ± 0.015	0.812 ± 0.005	0.232 ± 0.012	0.503 ± 0.006
	T3	0.282 ± 0.018	0.305 ± 0.018	0.198 ± 0.014	0.817 ± 0.005	0.227 ± 0.012	0.513 ± 0.005
	T4	0.260 ± 0.017	0.286 ± 0.018	0.179 ± 0.014	0.815 ± 0.004	0.217 ± 0.012	0.519 ± 0.005
	T5	0.284 ± 0.019	0.306 ± 0.018	0.201 ± 0.015	0.812 ± 0.004	0.228 ± 0.013	0.510 ± 0.005
	T6	0.278 ± 0.018	0.303 ± 0.018	0.197 ± 0.014	0.816 ± 0.004	0.228 ± 0.012	0.518 ± 0.005
	T7	0.284 ± 0.018	0.307 ± 0.018	0.202 ± 0.015	0.807 ± 0.005	0.231 ± 0.012	0.507 ± 0.005
	T8	0.307 ± 0.019	0.322 ± 0.019	0.219 ± 0.015	0.806 ± 0.005	0.237 ± 0.012	0.500 ± 0.005
InstructGPT	T1	0.265 ± 0.018	0.289 ± 0.018	0.187 ± 0.014	0.812 ± 0.005	0.221 ± 0.012	0.511 ± 0.005
	T2	0.276 ± 0.018	0.295 ± 0.018	0.189 ± 0.014	0.813 ± 0.005	0.219 ± 0.012	0.506 ± 0.006
	T3	0.263 ± 0.017	0.286 ± 0.018	0.183 ± 0.014	0.818 ± 0.005	0.216 ± 0.012	0.515 ± 0.005
	T4	0.261 ± 0.017	0.286 ± 0.018	0.184 ± 0.014	0.814 ± 0.004	0.218 ± 0.012	0.512 ± 0.005
	T5	0.274 ± 0.018	0.298 ± 0.018	0.193 ± 0.014	0.814 ± 0.005	0.223 ± 0.012	0.507 ± 0.005
	T6	0.278 ± 0.018	0.300 ± 0.018	0.194 ± 0.014	0.813 ± 0.004	0.223 ± 0.012	0.507 ± 0.005
	T7	0.270 ± 0.018	0.293 ± 0.018	0.189 ± 0.014	0.800 ± 0.005	0.219 ± 0.012	0.496 ± 0.006
	T8	0.264 ± 0.018	0.284 ± 0.018	0.189 ± 0.014	0.813 ± 0.004	0.216 ± 0.012	0.503 ± 0.005
Llama2-7B	T1	0.252 ± 0.018	0.280 ± 0.018	0.172 ± 0.014	0.814 ± 0.004	0.212 ± 0.012	0.515 ± 0.005
	T2	0.236 ± 0.017	0.260 ± 0.017	0.158 ± 0.013	0.819 ± 0.004	0.198 ± 0.012	0.506 ± 0.005
	T3	0.244 ± 0.017	0.272 ± 0.018	0.160 ± 0.014	0.818 ± 0.004	0.202 ± 0.012	0.512 ± 0.005
	T4	0.254 ± 0.017	0.283 ± 0.018	0.173 ± 0.014	0.812 ± 0.004	0.216 ± 0.012	0.516 ± 0.005
	T5	0.236 ± 0.017	0.269 ± 0.018	0.163 ± 0.014	0.806 ± 0.005	0.207 ± 0.012	0.513 ± 0.005
	T6	0.243 ± 0.017	0.272 ± 0.018	0.169 ± 0.014	0.806 ± 0.005	0.207 ± 0.012	0.512 ± 0.005
	T7	0.262 ± 0.017	0.283 ± 0.018	0.183 ± 0.014	0.809 ± 0.005	0.217 ± 0.012	0.506 ± 0.006
	T8	0.256 ± 0.018	0.274 ± 0.017	0.179 ± 0.014	0.812 ± 0.005	0.208 ± 0.012	0.505 ± 0.006
Llama2-13B	T1	0.255 ± 0.018	0.276 ± 0.018	0.180 ± 0.014	0.810 ± 0.004	0.213 ± 0.012	0.499 ± 0.006
	T2	0.168 ± 0.015	0.186 ± 0.015	0.120 ± 0.012	0.818 ± 0.004	0.160 ± 0.011	0.487 ± 0.005
	T3	0.226 ± 0.017	0.244 ± 0.017	0.163 ± 0.013	0.815 ± 0.005	0.196 ± 0.012	0.497 ± 0.006
	T4	0.257 ± 0.017	0.279 ± 0.018	0.182 ± 0.014	0.812 ± 0.005	0.216 ± 0.012	0.502 ± 0.006
	T5	0.257 ± 0.018	0.280 ± 0.018	0.179 ± 0.014	0.808 ± 0.005	0.212 ± 0.012	0.497 ± 0.006
	T6	0.247 ± 0.017	0.267 ± 0.018	0.171 ± 0.014	0.809 ± 0.005	0.204 ± 0.012	0.497 ± 0.006
	T7	0.266 ± 0.018	0.288 ± 0.018	0.185 ± 0.014	0.801 ± 0.005	0.215 ± 0.012	0.492 ± 0.006
	T8	0.195 ± 0.015	0.217 ± 0.016	0.145 ± 0.013	0.808 ± 0.005	0.180 ± 0.011	0.489 ± 0.005

Just Accepted

Table 12. Re-ranking performance for the Goodreads dataset. All metrics are computed with a cutoff of size $n = |RL| = 10$. For all metrics, the higher the scores, the better the performance. For each measure, we also report the confidence intervals calculated with a 95% confidence level.

		NDCG	α-NDCG	EILD	ILD	rSRecall	SRecall
MF		0.249 ± 0.022	0.271 ± 0.022	0.092 ± 0.010	0.689 ± 0.006	0.355 ± 0.025	0.871 ± 0.009
Re-ranker	**Template**						
Random		0.084 ± 0.010	0.109 ± 0.013	0.026 ± 0.006	0.702 ± 0.006	0.199 ± 0.021	0.889 ± 0.009
MMR		0.237 ± 0.020	0.253 ± 0.021	0.090 ± 0.011	0.810 ± 0.004	0.336 ± 0.025	0.934 ± 0.006
xQuad		0.229 ± 0.019	0.308 ± 0.024	0.084 ± 0.009	0.632 ± 0.006	0.386 ± 0.026	0.958 ± 0.005
RxQuad		0.229 ± 0.019	0.303 ± 0.024	0.084 ± 0.009	0.635 ± 0.006	0.381 ± 0.026	0.946 ± 0.006
ChatGPT	T1	0.161 ± 0.018	0.210 ± 0.022	0.045 ± 0.008	0.655 ± 0.007	0.269 ± 0.025	0.877 ± 0.009
	T2	0.180 ± 0.018	0.227 ± 0.022	0.059 ± 0.009	0.672 ± 0.007	0.296 ± 0.025	0.886 ± 0.008
	T3	0.156 ± 0.017	0.209 ± 0.022	0.046 ± 0.007	0.652 ± 0.007	0.266 ± 0.024	0.883 ± 0.009
	T4	0.149 ± 0.017	0.204 ± 0.022	0.039 ± 0.007	0.650 ± 0.007	0.261 ± 0.024	0.884 ± 0.008
	T5	0.188 ± 0.019	0.242 ± 0.023	0.061 ± 0.009	0.662 ± 0.007	0.318 ± 0.026	0.893 ± 0.008
	T6	0.174 ± 0.018	0.234 ± 0.023	0.052 ± 0.008	0.652 ± 0.007	0.302 ± 0.025	0.903 ± 0.008
	T7	0.153 ± 0.018	0.207 ± 0.022	0.045 ± 0.008	0.644 ± 0.007	0.273 ± 0.025	0.880 ± 0.008
	T8	0.163 ± 0.017	0.218 ± 0.022	0.053 ± 0.008	0.658 ± 0.007	0.287 ± 0.025	0.885 ± 0.008
InstructGPT	T1	0.114 ± 0.015	0.171 ± 0.021	0.029 ± 0.006	0.658 ± 0.006	0.229 ± 0.024	0.879 ± 0.008
	T2	0.125 ± 0.017	0.179 ± 0.022	0.037 ± 0.007	0.664 ± 0.006	0.235 ± 0.024	0.889 ± 0.008
	T3	0.119 ± 0.016	0.173 ± 0.021	0.034 ± 0.007	0.658 ± 0.007	0.229 ± 0.025	0.882 ± 0.009
	T4	0.109 ± 0.015	0.163 ± 0.021	0.028 ± 0.006	0.659 ± 0.006	0.225 ± 0.024	0.882 ± 0.008
	T5	0.123 ± 0.016	0.184 ± 0.022	0.033 ± 0.007	0.652 ± 0.006	0.241 ± 0.024	0.887 ± 0.008
	T6	0.122 ± 0.016	0.180 ± 0.021	0.033 ± 0.006	0.652 ± 0.006	0.244 ± 0.025	0.892 ± 0.008
	T7	0.105 ± 0.014	0.159 ± 0.020	0.024 ± 0.005	0.653 ± 0.006	0.221 ± 0.024	0.870 ± 0.009
	T8	0.109 ± 0.016	0.163 ± 0.021	0.031 ± 0.007	0.655 ± 0.006	0.223 ± 0.025	0.884 ± 0.008
Llama2-7B	T1	0.117 ± 0.015	0.160 ± 0.020	0.030 ± 0.006	0.674 ± 0.006	0.215 ± 0.023	0.885 ± 0.009
	T2	0.100 ± 0.013	0.143 ± 0.019	0.025 ± 0.006	0.675 ± 0.006	0.200 ± 0.022	0.875 ± 0.009
	T3	0.084 ± 0.012	0.113 ± 0.015	0.021 ± 0.005	0.681 ± 0.006	0.176 ± 0.020	0.879 ± 0.009
	T4	0.118 ± 0.015	0.156 ± 0.019	0.026 ± 0.006	0.678 ± 0.006	0.215 ± 0.022	0.892 ± 0.008
	T5	0.111 ± 0.014	0.142 ± 0.017	0.035 ± 0.006	0.673 ± 0.007	0.210 ± 0.022	0.876 ± 0.009
	T6	0.112 ± 0.014	0.146 ± 0.017	0.033 ± 0.006	0.674 ± 0.007	0.209 ± 0.021	0.872 ± 0.009
	T7	0.094 ± 0.013	0.122 ± 0.016	0.031 ± 0.007	0.693 ± 0.006	0.200 ± 0.022	0.880 ± 0.009
	T8	0.070 ± 0.009	0.094 ± 0.012	0.022 ± 0.005	0.699 ± 0.006	0.173 ± 0.020	0.888 ± 0.009
Llama2-13B	T1	0.110 ± 0.015	0.159 ± 0.020	0.027 ± 0.006	0.670 ± 0.006	0.222 ± 0.023	0.890 ± 0.009
	T2	0.104 ± 0.013	0.154 ± 0.015	0.031 ± 0.006	0.666 ± 0.007	0.222 ± 0.022	0.887 ± 0.009
	T3	0.106 ± 0.012	0.160 ± 0.015	0.026 ± 0.005	0.666 ± 0.006	0.228 ± 0.020	0.886 ± 0.009
	T4	0.113 ± 0.015	0.169 ± 0.019	0.028 ± 0.006	0.667 ± 0.006	0.234 ± 0.021	0.891 ± 0.008
	T5	0.084 ± 0.014	0.123 ± 0.017	0.024 ± 0.005	0.665 ± 0.006	0.190 ± 0.022	0.873 ± 0.008
	T6	0.080 ± 0.014	0.120 ± 0.019	0.021 ± 0.006	0.668 ± 0.007	0.180 ± 0.022	0.880 ± 0.008
	T7	0.091 ± 0.012	0.116 ± 0.015	0.026 ± 0.006	0.687 ± 0.006	0.180 ± 0.020	0.883 ± 0.009
	T8	0.082 ± 0.011	0.116 ± 0.015	0.026 ± 0.006	0.696 ± 0.006	0.196 ± 0.021	0.880 ± 0.009

Just Accepted

Received 15 January 2024; revised 22 August 2024; accepted 30 September 2024

Just Accepted